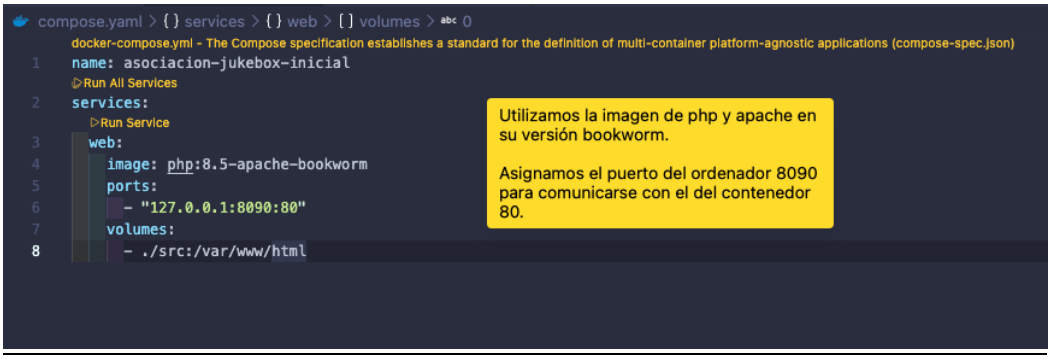
Práctica acumulativa PHP RA1 – RA2 (DIVERTIDA)

1) CREACIÓN DE LA ESTRUCTURA BASE DEL PROYECTO

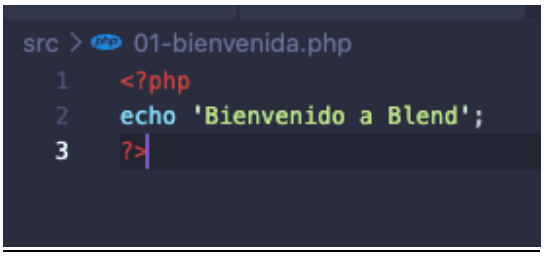
Creareamos una carpeta llamada “Asociación-Blend-PHP” y abriremos el proyecto en VSCode, creando la siguiente estructura.



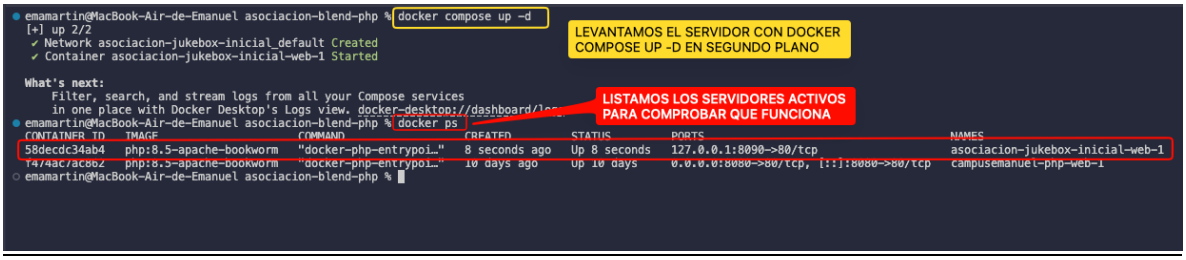
Creamos nuestro compose.yaml

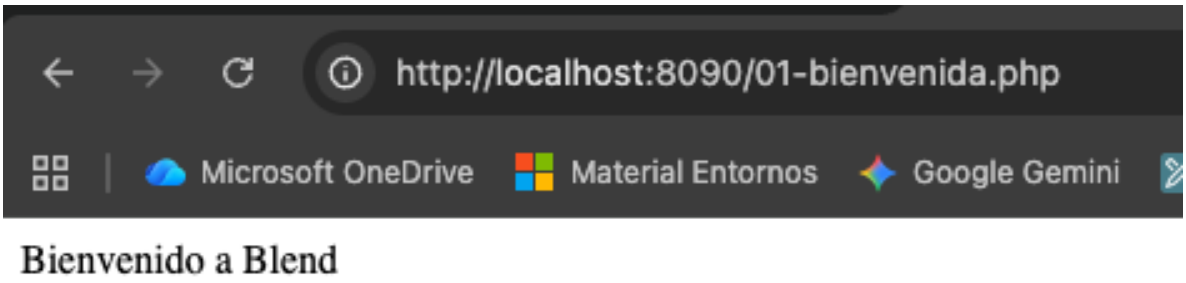


Añadimos una página inicial para comprobar el funcionamiento



Arrancamos el servidor y comprobamos en el navegador que todo funciona





Esta página en programación actúa como “prueba de vida o sanity check”. Como acabamos de crear una carpeta nueva y hemos configurado el servidor en un puerto diferente al habitual (8090), antes de escribir código complejo con variables, HTML, matemáticas, etc, necesitamos saber si la infraestructura funciona.

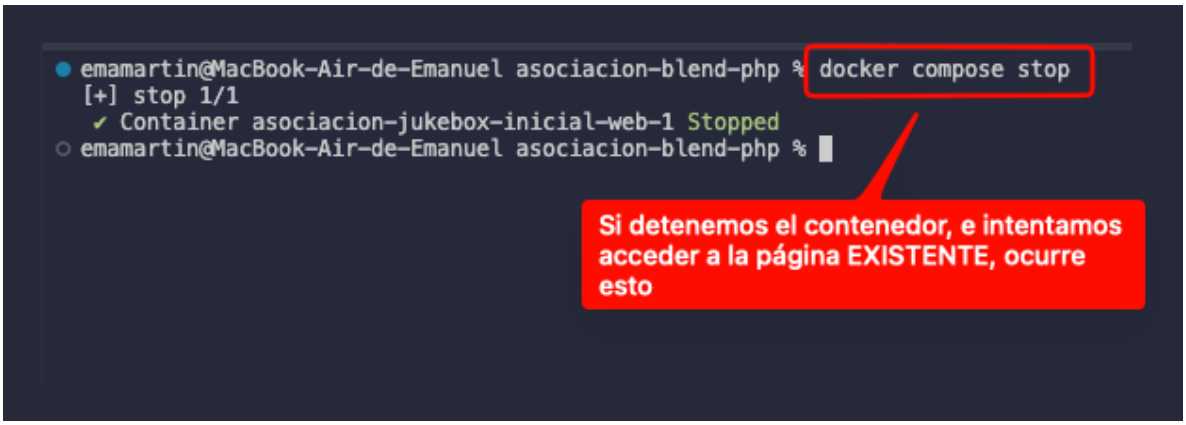
COMPROBACIONES FORZANDO ERRORES

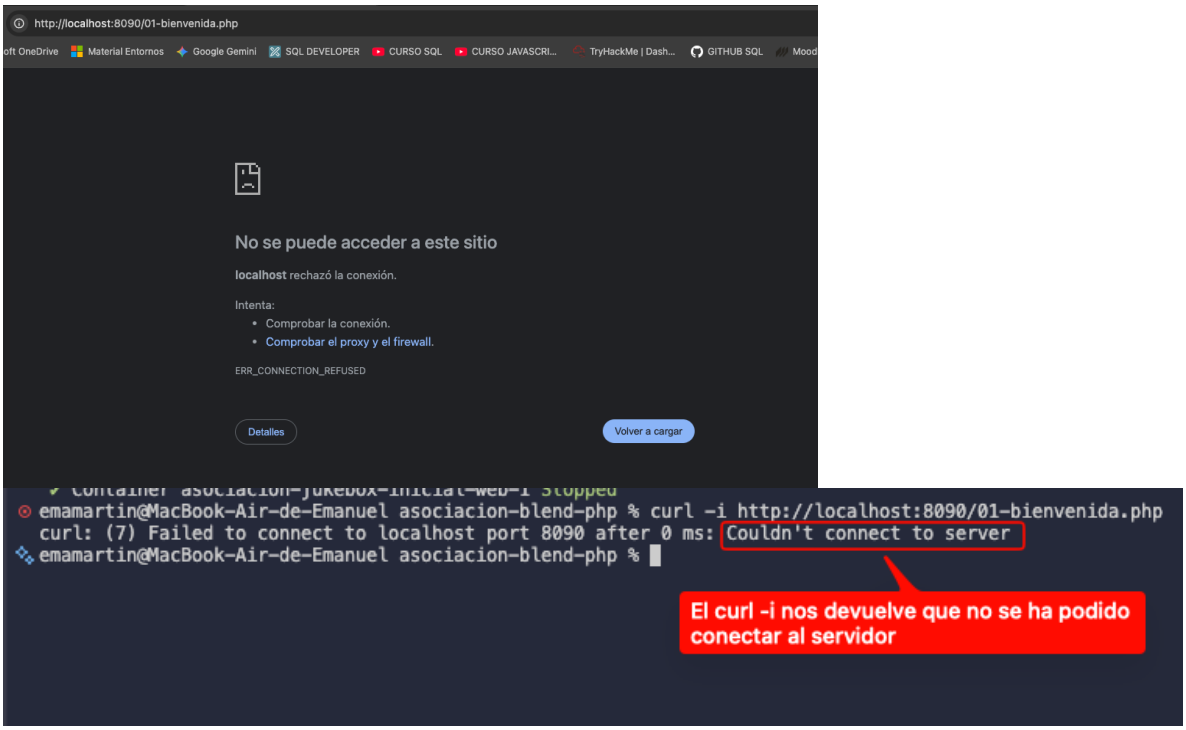
- BUSCANDO UNA PÁGINA INEXISTENTE





- LA PÁGINA EXISTE PERO EL SERVIDOR ESTÁ DETENIDO





- RESTAURAMOS LA CONEXIÓN AL SERVIDOR



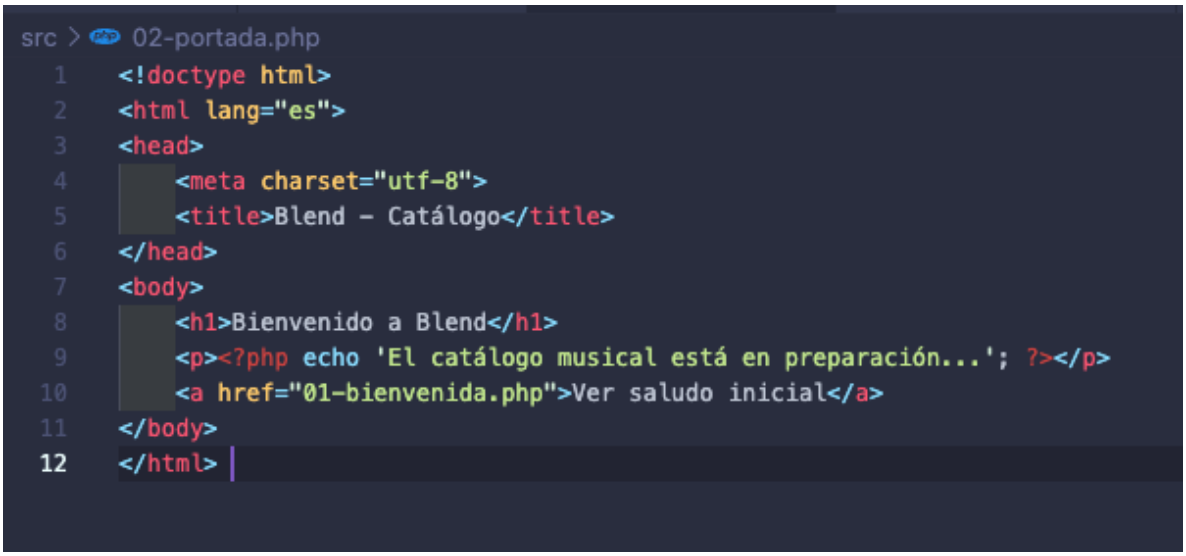
2) CREANDO PORTADA Y DEFINIENDO LA FICHA DE LA PRIMERA CANCIÓN

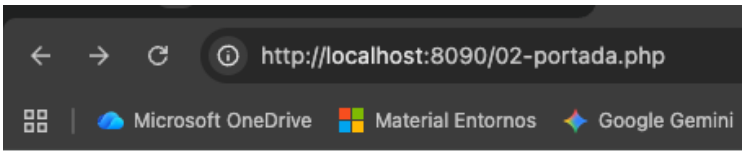
Maquetamos la base de lo que próximamente será el catálogo de Blend, utilizando variables para los datos de la canción, y preparando su salida segura (mediante HTMLSPECIALCHARS).

El objetivo es empezar a maquetar cómo se rverá una sola canción de nuestro reproductor BLEND en la pantalla, pero preparando el terreno para que los datos sean dinámicos.

CREACIÓN DE LA PORTADA

Creamos una página HTML básica que da la bienvenida a la aplicación. A través de esta página demostramos que un archivo .php puede contener un 99% de HTML normal y corriente. PHP actúa solo como invitado, interviniendo cuando le abrimos la puerta con la etiqueta <?php.





Bienvenido a Blend

El catálogo musical está en preparación...

Ver saludo inicial

Con esto podemos comprobar que no aprenderemos ningún lenguaje visual nuevo. Todo el código HTML y CSS que hemos hecho en la práctica de Desarrollo Cliente, funcionará perfectamente dentro de archivos .php.

CREACIÓN DE LA FICHA DE LA CANCIÓN

En este caso definimos los datos de una canción de nuestra lista, usando como ejemplo una de las canciones del reproductor musical BLEND implementado en Desarrollo Cliente, y empleando para ello distintos tipos de variables (string,number,boolean) y los imprimiremos de forma segura utilizando htmlspecialchars.

Aquí entrará la verdadera lógica, dado que esta estructura se creará con el fin de evitar escribir a mano cada uno de los datos (título, artista, duración) de las 21 canciones que tiene el reproductor musical en el archivo playlist.json. En su lugar crearemos variables, que luego veremos que almacenarán esta información dinámicamente.

src > 03-cancion.php

```php
<?php
const APP_NAME = 'Blend - Tu experiencia musical';

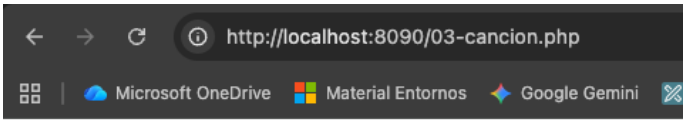
// DATOS DE LA CANCIÓN (Variables)
$title = 'Indie Rock Dreamy'; // string
$artist = 'Alex Morgan';      // string
$duration = 185;              // int (Segundos enteros)
$volume = 0.5;                // float
$available = true;            // bool

// SEGURIDAD (Escapar el título por si tuviera caracteres raros)
$safeTitle = htmlspecialchars($title, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
?>

<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title><?= APP_NAME ?></title>
</head>
<body>
    <h1><?= APP_NAME ?></h1>

    <h2>Ficha de la canción:</h2>
    <ul>
        <li><strong>Título:</strong> <?= $safeTitle ?></li>
        <li><strong>Artista:</strong> <?= $artist ?></li>
        <li><strong>Duración (segundos):</strong> <?= $duration ?></li>
    </ul>

    <!-- Creamos un inspeccionador temporal para comprobar los tipos de datos -->
    <h3>Inspección de tipos (var_dump temporal):</h3>
    <pre>
        <?php
        echo "Duración: ";
        var_dump($duration);
        echo "Volumen: ";
        var_dump($volume);
        ?>
    </pre>
</body>
</html>
```



Blend - Tu experiencia musical

Ficha de la canción:

- **Título:** Indie Rock Dreamy
- **Artista:** Alex Morgan
- **Duración (segundos):** 185

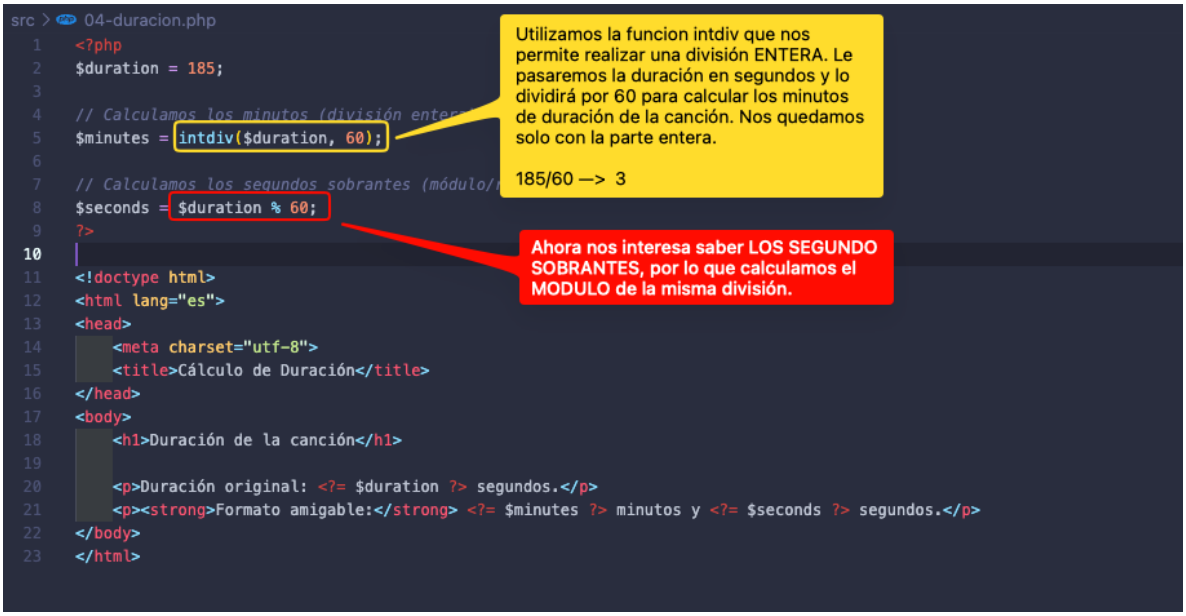
Inspección de tipos (var_dump temporal):

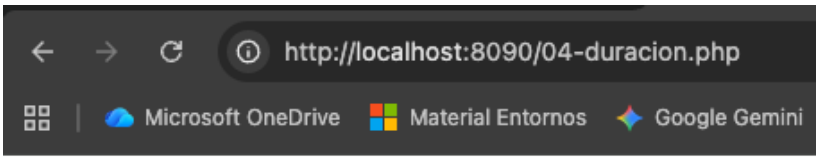
```
        Duración: int(185)
Volumen: float(0.5)
```

3) CÁLCULOS Y DECISIONES SOBRE UNA CANCIÓN

Aprenderemos a realizar cálculos a través de PHP para determinar la duración de una canción en minutos y segundos, a partir de los segundos totales de su duración.

Esto tiene todo el sentido del mundo, dado que para el usuario (admin) que utilizará esta sección no será para nada natural saber la duración de una canción en segundos.





Duración de la canción

Duración original: 185 segundos.

Formato amigable: 3 minutos y 5 segundos.

Ahora le pediremos a PHP que DECIDA en base a la duración de una canción.

Estableceremos tres mensajes posibles:

- Sin duración (Si la duración de la canción es 0)

- Hasta cuatro minutos (Si la duración de la canción es menor o igual a 240 segundos, es decir 4 minutos).
- Más de cuatro minutos (Si la duración de la canción es mayor a 240 segundos, es decir 4 minutos).

Para ello utilizaremos estructuras condicionales IF/ELSEIF Y ELSE.

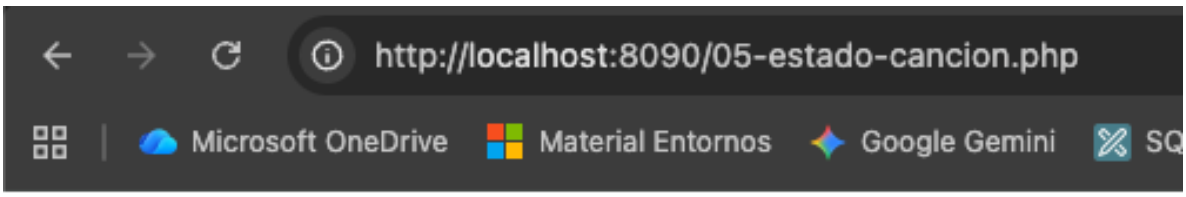
src > 05-estado-cancion.php

```php
<?php
$duration = 185;

if ($duration === 0) {
    $estado = 'Sin duración';
} elseif ($duration <= 240) {
    $estado = 'Hasta cuatro minutos';
} else {
    $estado = 'Más de cuatro minutos';
}
?>

<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Estado de la canción</title>
</head>
<body>
    <h1>Clasificación de la pista</h1>
    <p>Duración: <?= $duration ?> segundos.</p>
    <p>Estado automático: <strong><?= $estado ?></strong></p>
</body>
</html>
```

Como sabemos ya de sobra, las estructuras condicionales IF/ELSEIF/ELSE son EXCLUYENTES, por lo que la ejecución (condición verdadera) de una de ellas no permite la ejecucón de las demás.

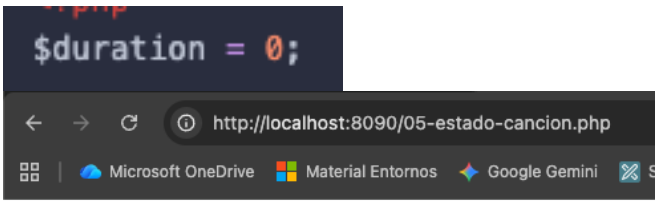


Clasificación de la pista

Duración: 185 segundos.

Estado automático: **Hasta cuatro minutos**

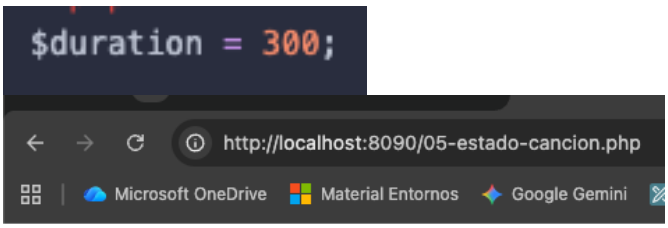
Si modificamos la duración comprobamos las diferentes salidas:



Clasificación de la pista

Duración: 0 segundos.

Estado automático: **Sin duración**



Clasificación de la pista

Duración: 300 segundos.

Estado automático: **Más de cuatro minutos**

4) CONSTRUCCIÓN DEL CATÁLOGO MUSICAL

Desde ahora nuestro reproductor comenzará a cobrar vida en el servidor. Nos enfocaremos en el catálogo. Si tuviéramos que crear la página del catálogo copiando y pegando HTML de cada canción, 10,20,30, o la cantidad de veces que sea necesario según la cantidad de canciónes, el código HTML sería totalmente inmanejable.

Lo construimos dinámicamente, utilizando:

- Bucles
- Arrays asociativos

Para ello, haremos primero una prueba para probar su funcionamiento:

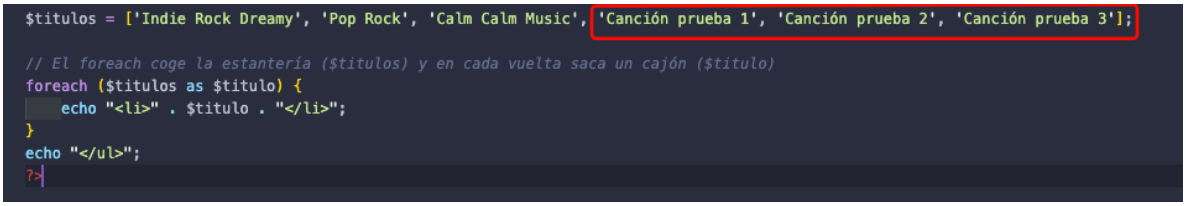
ANTES de cargar todas las canciones de golpe al catálogo, practicaremos la mecánica de los bucles:

- For
- While
- Foreach



De esta manera, mantenemos nuestro HTML simple, y si queremos añadir o quitar elementos de la lista no lo hacemos directamente en el HTML, sino modificando las condiciones de los bucles.

El foreach es muy útil ya que es automático, si yo quisiera añadir una canción al array y que salga mostrada, SOLO TENGO QUE AÑADIRLA LA ARRAY.



Lista con FOREACH

- Indie Rock Dreamy
- Pop Rock
- Calm Calm Music
- Canción prueba 1
- Canción prueba 2
- Canción prueba 3

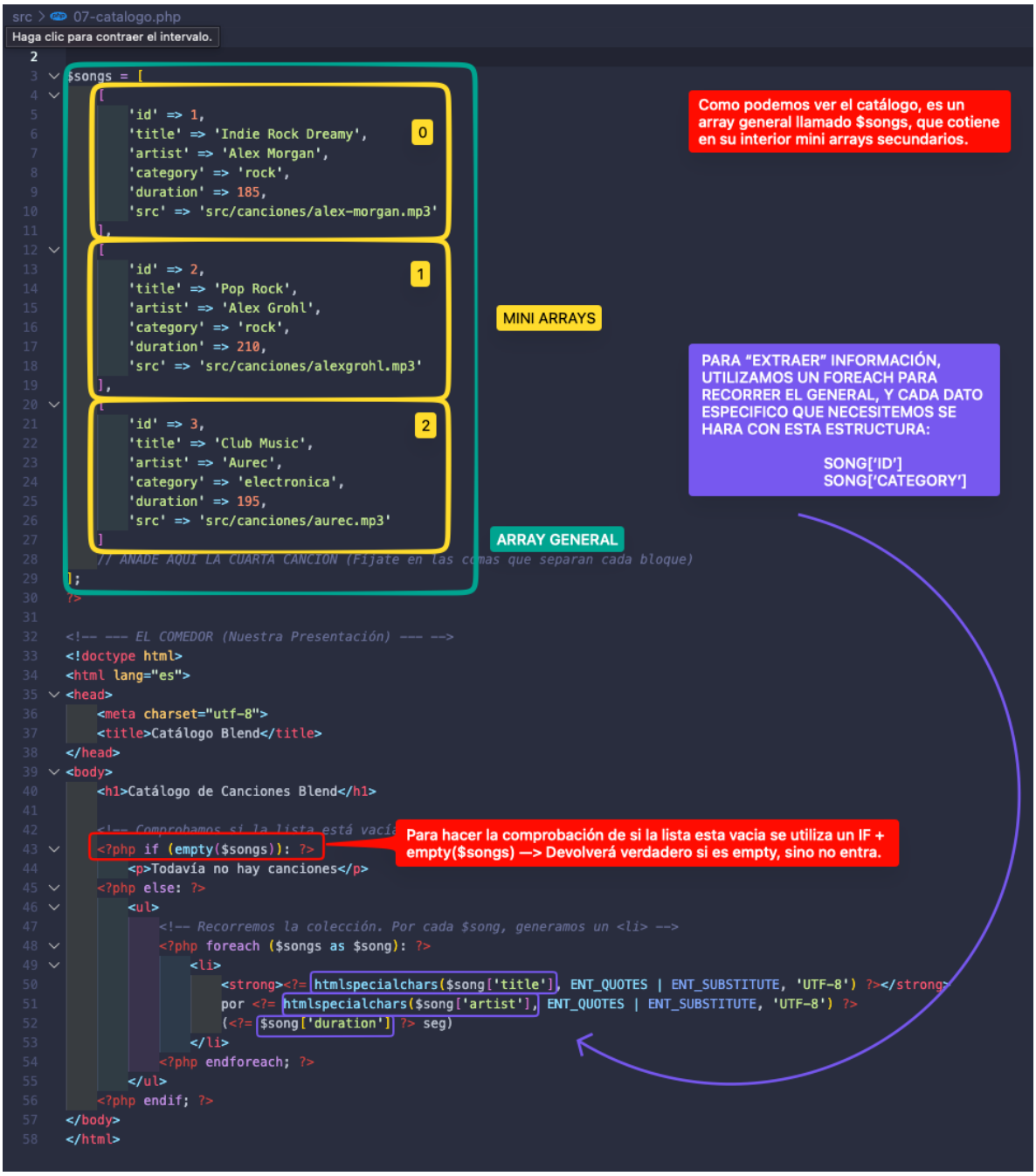
CATÁLOGO REAL

En base a los bucles que hemos visto anteriormente, ahora construiremos la estructura de datos para nuestro catálogo musical. Si tomamos como referencia que en la asignatura Desarrollo Cliente, al crear el reproductor BLEND utilizamos un archivo playlist.json, veremos que esta estructura se parece mucho a ese archivo.

Hasta el momento, utilzamos el foreach para recorrer un array simple que solo contenía el título de las canciones. Sin embargo, una canción no es solo un título, sino un conjunto de datos:

→ Id
→ Titulo
→ Artista
→ Duración

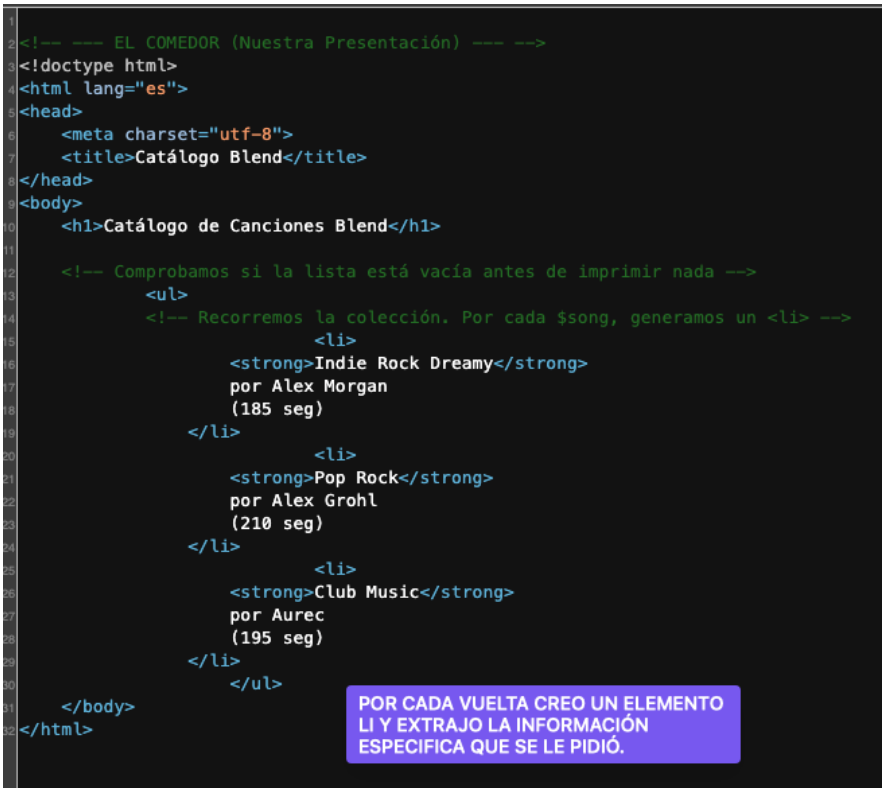
En PHP esto se denomina **ARRAY ASOCIATIVO**



Catálogo de Canciones Blend

- **Indie Rock Dreamy** por Alex Morgan (185 seg)
- **Pop Rock** por Alex Grohl (210 seg)
- **Club Music** por Aurec (195 seg)

Si vemos el código fuente podemos ver lo que ha generado PHP a nivel HTML:




```
<!-- --- EL COMEDOR (Nuestra Presentación) --- -->
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Catálogo Blend</title>
</head>
<body>
    <h1>Catálogo de Canciones Blend</h1>

    <!-- Comprobamos si la lista está vacía antes de imprimir nada -->
            <ul>
            <!-- Recorremos la colección. Por cada $song, generamos un <li> -->
                        <li>
                    <strong>Indie Rock Dreamy</strong>
                    por Alex Morgan
                    (185 seg)
                </li>
                        <li>
                    <strong>Pop Rock</strong>
                    por Alex Grohl
                    (210 seg)
                </li>
                        <li>
                    <strong>Club Music</strong>
                    por Aurec
                    (195 seg)
                </li>
                    </ul>
    </body>
</html>
```

Si añadimos una canción al catálogo, automáticamente se verá reflejada en el HTML.

```php
$songs = [
    [
        'id' => 1,
        'title' => 'Indie Rock Dreamy',
        'artist' => 'Alex Morgan',
        'category' => 'rock',
        'duration' => 185,
        'src' => 'src/canciones/alex-morgan.mp3'
    ],
    [
        'id' => 2,
        'title' => 'Pop Rock',
        'artist' => 'Alex Grohl',
        'category' => 'rock',
        'duration' => 210,
        'src' => 'src/canciones/alexgrohl.mp3'
    ],
    [
        'id' => 3,
        'title' => 'Calm Calm Music',
        'artist' => 'Aurec',
        'category' => 'relax',
        'duration' => 240,
        'src' => 'src/canciones/aurec2.mp3'
    ],
    [
        'id' => 4,
        'title' => 'Club Music',
        'artist' => 'Aurec',
        'category' => 'electronica',
        'duration' => 195,
        'src' => 'src/canciones/aurec.mp3'
    ],

    // AÑADE AQUÍ LA CUARTA CANCIÓN (Fíjate en las comas que separan cada bloque)
];
?>
```

Catálogo de Canciones Blend

- **Indie Rock Dreamy** por Alex Morgan (185 seg)
- **Pop Rock** por Alex Grohl (210 seg)
- **Calm Calm Music** por Aurec (240 seg)
- **Club Music** por Aurec (195 seg)

5) SEPARACIÓN DE FUNCIONES Y DATOS

Anteriormente realizamos la conversión de los segundos de una canción a minutos/segundos para mejorar su interpretación mediante una formula matemática.

Sin embargo, ¿qué ocurriría si este dato, tuviésemos que mostrarlo en lugares diferentes?

Supongamos que nuestro sitio web tiene:

- Página del catálogo
- Página de portada
- Página del perfil

Y en las tres, queremos mostrar la duración formateada.

Si lo seguimos trabajando como hasta ahora, tendríamos que copiar y pegar la fórmula matemática en cada uno de los tres archivos y cualquier cambio que queramos hacer (por ej., cambiar la palabra “minutos” por “min”, tendríamos que hacerlo en los tres archivos y esto no es práctico.

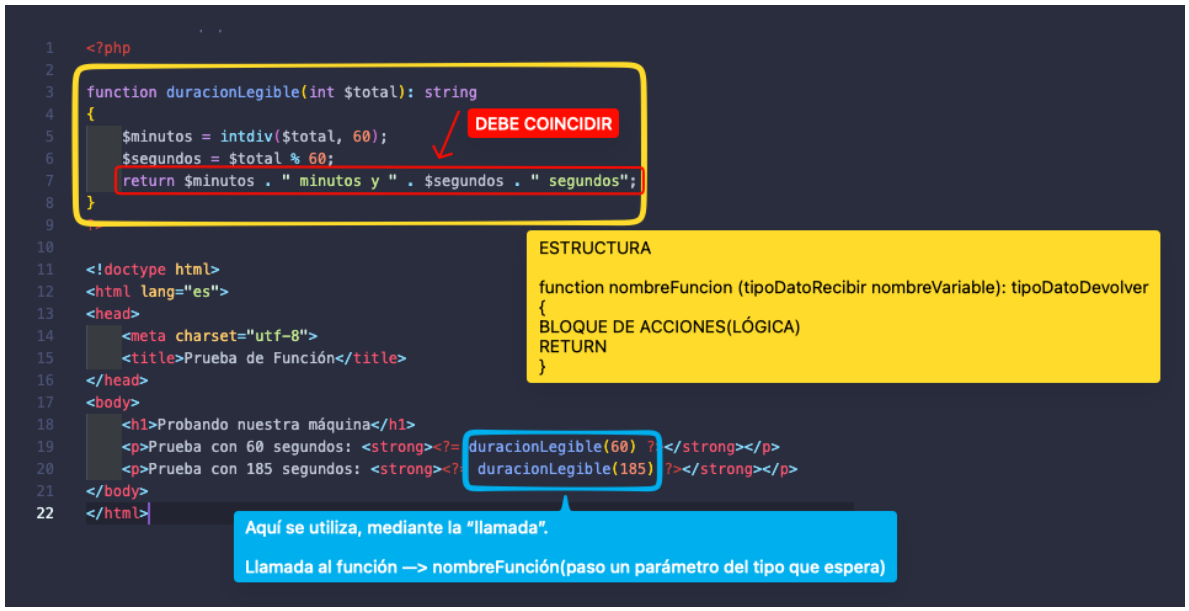
Para solucionar este problema utilizaremos → FUNCIONES

La función es una “receta reutilizable” que se escribe una sola vez y se la guarda en archivo común (librería) que todas las páginas pueden importar.

CREACIÓN DE LA FUNCIÓN

Una función podría asemejarse a una “máquina exprimidora”

- Entra una fruta (parámetro de entrada)
- Hace su trabajo interno (lógica)
- Devuelve un vaso de zumo (return)



Probando nuestra máquina

Prueba con 60 segundos: **1 minutos y 0 segundos**

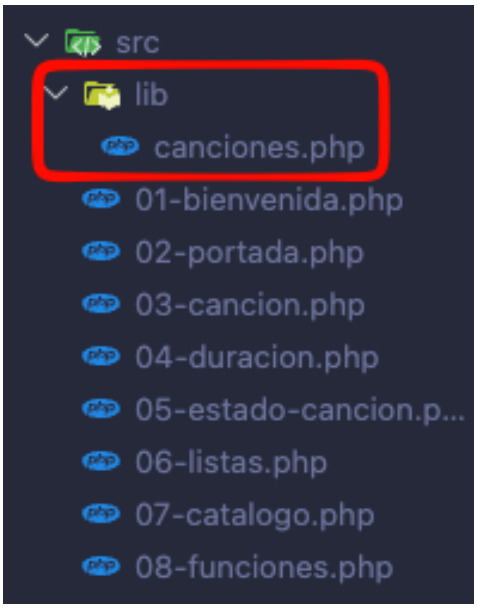
Prueba con 185 segundos: **3 minutos y 5 segundos**

SIN EMBARGO, DE ESTA MANERA TENEMOS UN INCONVENIENTE:

> La función duraciónLegible() SOLO EXISTE en el archivo 08-funciones.php. Si la quiero reutilizar en otra de mis páginas, no puedo hacerlo y no es eso lo que buscamos.

Para poder cumplir nuestro propósito, SACAREMOS la función y la guardaremos en un archivo externo que actuará como librería común.

Dentro de la carpeta donde tenemos nuestros archivos .php, crearemos una subcarpeta llamada lib y dentro de ella, un archivo llamado, en este caso, canciones.php.



Dentro de este nuevo archivo, cortamos y pegamos declaración de la función.



¿Qué está ocurriendo ahora? → El archivo 08-funciones.php ahora está realizando una llamada a una función que DESCONOCE, porque aún no se ha conectado con esta librería



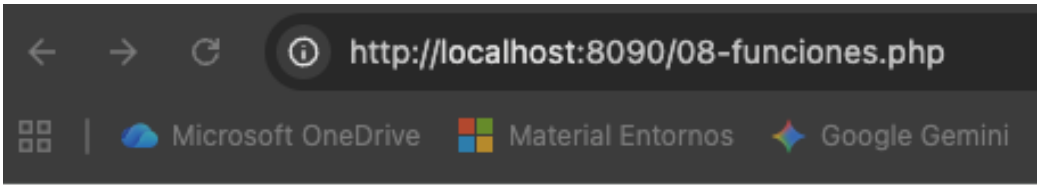


CONECTANDO LOS CABLES CON REQUIRE_ONCE

Lo que debemos hacer es decirle a 08-funciones.php QUE BUSQUE LA FUNCIÓN EN UN ARCHIVO EXTERNO.



Al ejecutar la página, vemos que la función hace su tarea correctamente pese a no estar declarada allí mismo:



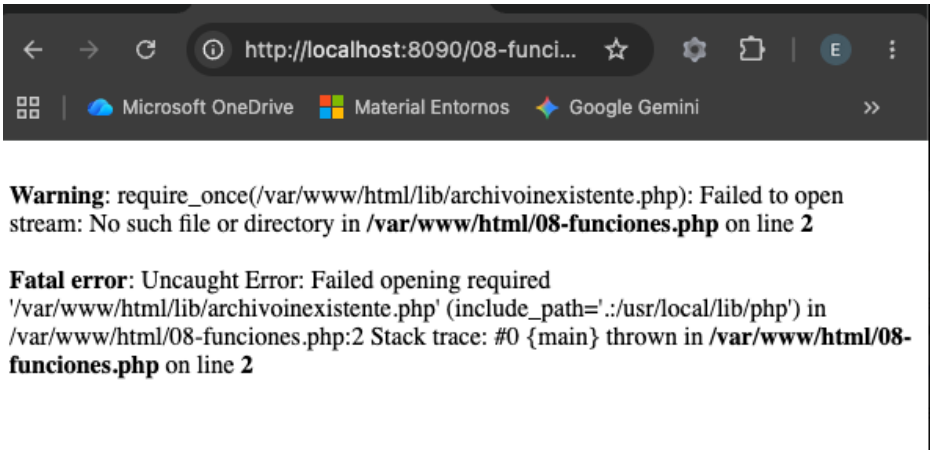
Probando nuestra máquina

Prueba con 60 segundos: **1 minutos y 0 segundos**

Prueba con 185 segundos: **3 minutos y 5 segundos**

Podemos verificar que esto es gracias a los conectores, dado que si cambiamos el archivo determinado en el required_once, la página se romperá.





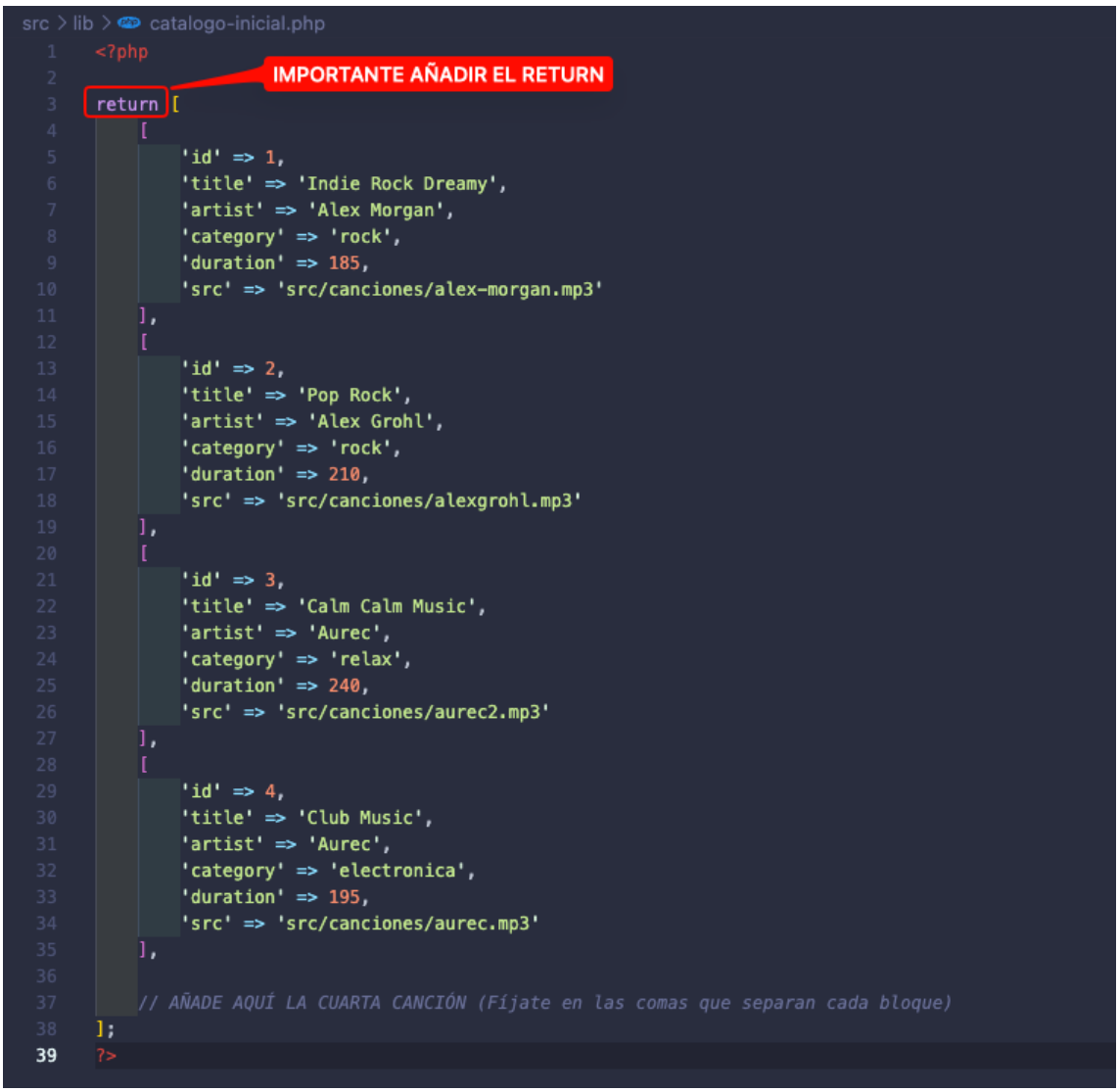
TRASLADAR EL CATÁLOGO DE CANCIONES A UN ARCHIVO EXTERNO

Tal y como hemos hecho con las funciones para poder reutilizarlas en diferentes archivos, trasladaremos el listado de canciones. Esto nos permitirá que si el día de mañana queremos hacer un filtro, buscar una canción o añadir una nueva, no tengamos esos datos atrapados en un HTML sino tener una "base de datos centralizada".

Para ello, dentro de la carpeta lib, creamos otro archivo llamado catalogo-inicial.php. Cortaremos el array $songs del archivo 07-catalogo.php y lo pegaremos en este archivo nuevo.

NOTA IMPORTANTE:

Para que este archivo catalogo-inicial.php DEVUELVA el array a quien se lo pida, añadiremos la palabra RETURN.




```php
<?php

return [
    [
        'id' => 1,
        'title' => 'Indie Rock Dreamy',
        'artist' => 'Alex Morgan',
        'category' => 'rock',
        'duration' => 185,
        'src' => 'src/canciones/alex-morgan.mp3'
    ],
    [
        'id' => 2,
        'title' => 'Pop Rock',
        'artist' => 'Alex Grohl',
        'category' => 'rock',
        'duration' => 210,
        'src' => 'src/canciones/alexgrohl.mp3'
    ],
    [
        'id' => 3,
        'title' => 'Calm Calm Music',
        'artist' => 'Aurec',
        'category' => 'relax',
        'duration' => 240,
        'src' => 'src/canciones/aurec2.mp3'
    ],
    [
        'id' => 4,
        'title' => 'Club Music',
        'artist' => 'Aurec',
        'category' => 'electronica',
        'duration' => 195,
        'src' => 'src/canciones/aurec.mp3'
    ],

    // AÑADE AQUÍ LA CUARTA CANCIÓN (Fíjate en las comas que separan cada bloque)
];
?>
```

Y ahora, al igual que hicimos con la funcion, tenemos que CONECTAR el archivo que va a utilizar el catálogo con este archivo externo que lo contiene:

src > 07-catalogo.php

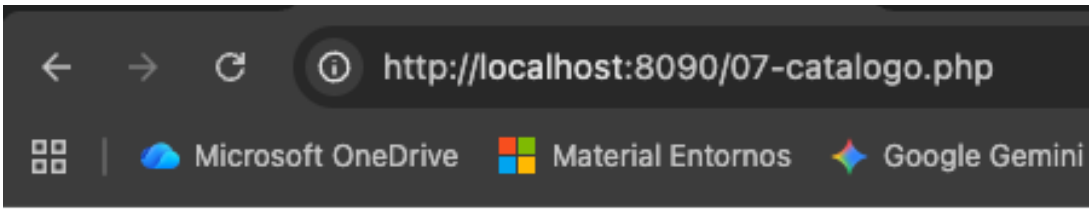
```php
<?php
$songs = require __DIR__ . '/lib/catalogo-inicial.php';
?>

<!-- --- EL COMEDOR (Nuestra Presentación) --- -->
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Catálogo Blend</title>
</head>
<body>
    <h1>Catálogo de Canciones Blend</h1>

    <!-- Comprobamos si la lista está vacía antes de imprimir nada -->
    <?php if (empty($songs)): ?>
        <p>Todavía no hay canciones</p>
    <?php else: ?>
        <ul>
            <!-- Recorremos la colección. Por cada $song, generamos un <li> -->
            <?php foreach ($songs as $song): ?>
                <li>
                    <strong><?= htmlspecialchars($song['title'], ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8') ?></strong>
                    por <?= htmlspecialchars($song['artist'], ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8') ?>
                    (<?= $song['duration'] ?> seg)
                </li>
            <?php endforeach; ?>
        </ul>
    <?php endif; ?>
</body>
</html>
```

Le dice, en $songs RECIBIRÉ lo que buscaré en el directorio actual, entrando al a carpeta LIB y al archivo catalogo-inicial.php

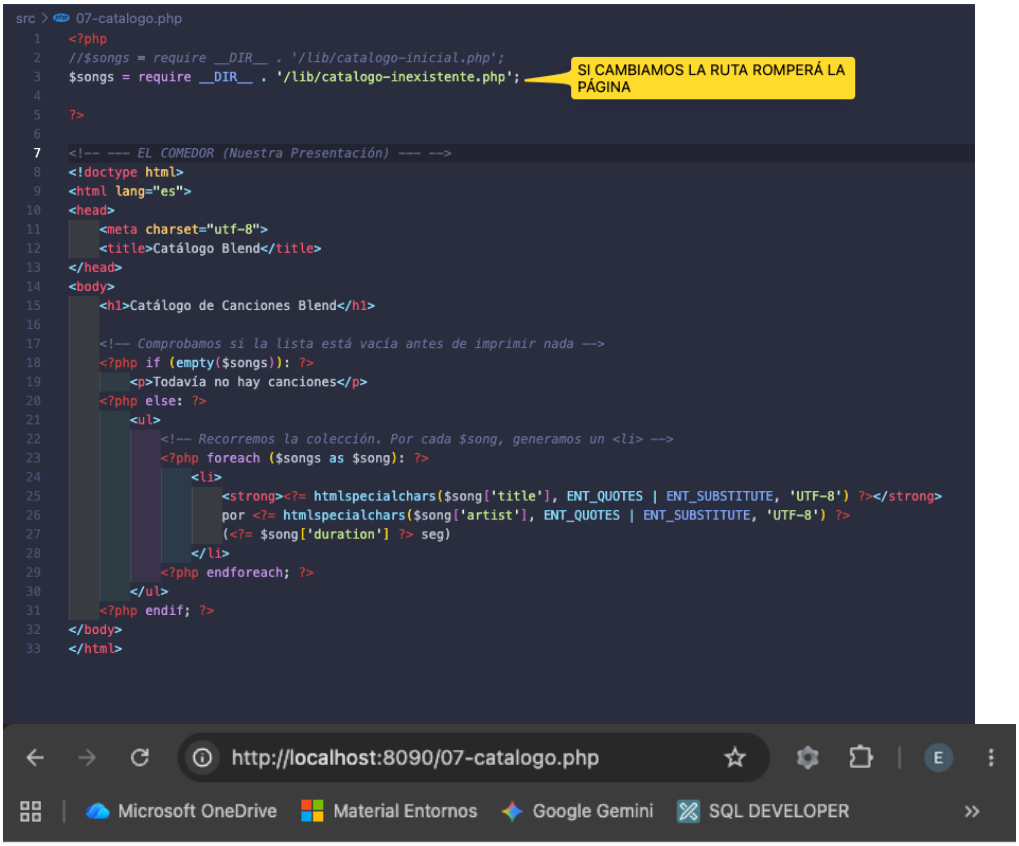
Si cargamos la página, recibirá los datos correctamente y se mostrará igual que antes:



Catálogo de Canciones Blend

- **Indie Rock Dreamy** por Alex Morgan (185 seg)
- **Pop Rock** por Alex Grohl (210 seg)
- **Calm Calm Music** por Aurec (240 seg)
- **Club Music** por Aurec (195 seg)

Si modificamos esa ruta en la que tiene que ir a buscar el catálogo, se romperá la página



Warning: require(/var/www/html/lib/catalogo-inexistente.php): Failed to open stream: No such file or directory in **/var/www/html/07-catalogo.php** on line **3**

Fatal error: Uncaught Error: Failed opening required '/var/www/html/lib/catalogo-inexistente.php' (include_path='.:/usr/local/lib/php') in /var/www/html/07-catalogo.php:3 Stack trace: #0 {main} thrown in **/var/www/html/07-catalogo.php** on line **3**

6) INTERACTUANDO CON EL USUARIO MEDIANTE LA URL

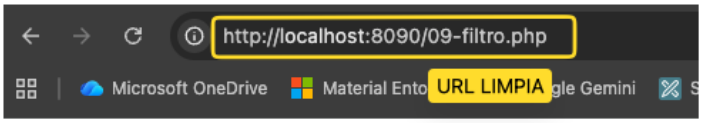
En este bloque le enseñaremos a nuestra web, a escuchar al usuario.

En PHP, cuando un usuario escribe algo en la URL después de un signo de interrogación, (ej: ?category=rock), **PHP LO ATRAPA AUTOMÁTICAMENTE** y lo guarda en una "caja especial" llamada $_GET.

En este punto, crearemos una página que → Filtre canciones por categoría, leyendo la URL.



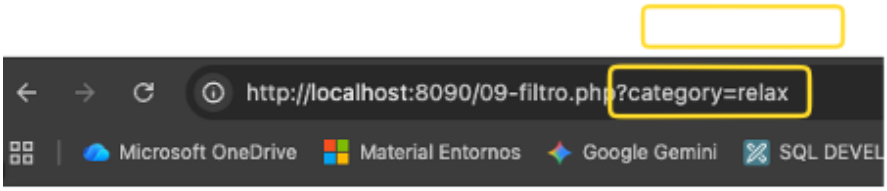
PRUEBAS:



Catálogo filtrado por: todas

- **Indie Rock Dreamy** (rock)
- **Pop Rock** (rock)
- **Calm Calm Music** (relax)
- **Club Music** (electronica)

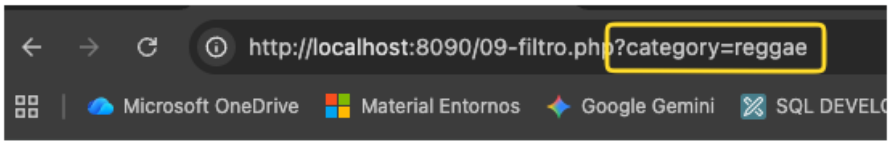
EN ESTE CASO, SE EJECUTA EL ?? 'TODAS'
y category='todas'



Catálogo filtrado p

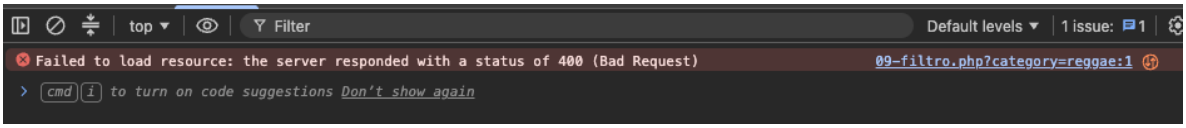
- **Calm Calm Music** (relax)

EL VALOR DE $_GET='RELAX' por lo que se asigna ese valor a $category y se filtra correctamente solo la de categoria relax.



Error 400: Categoría no válida

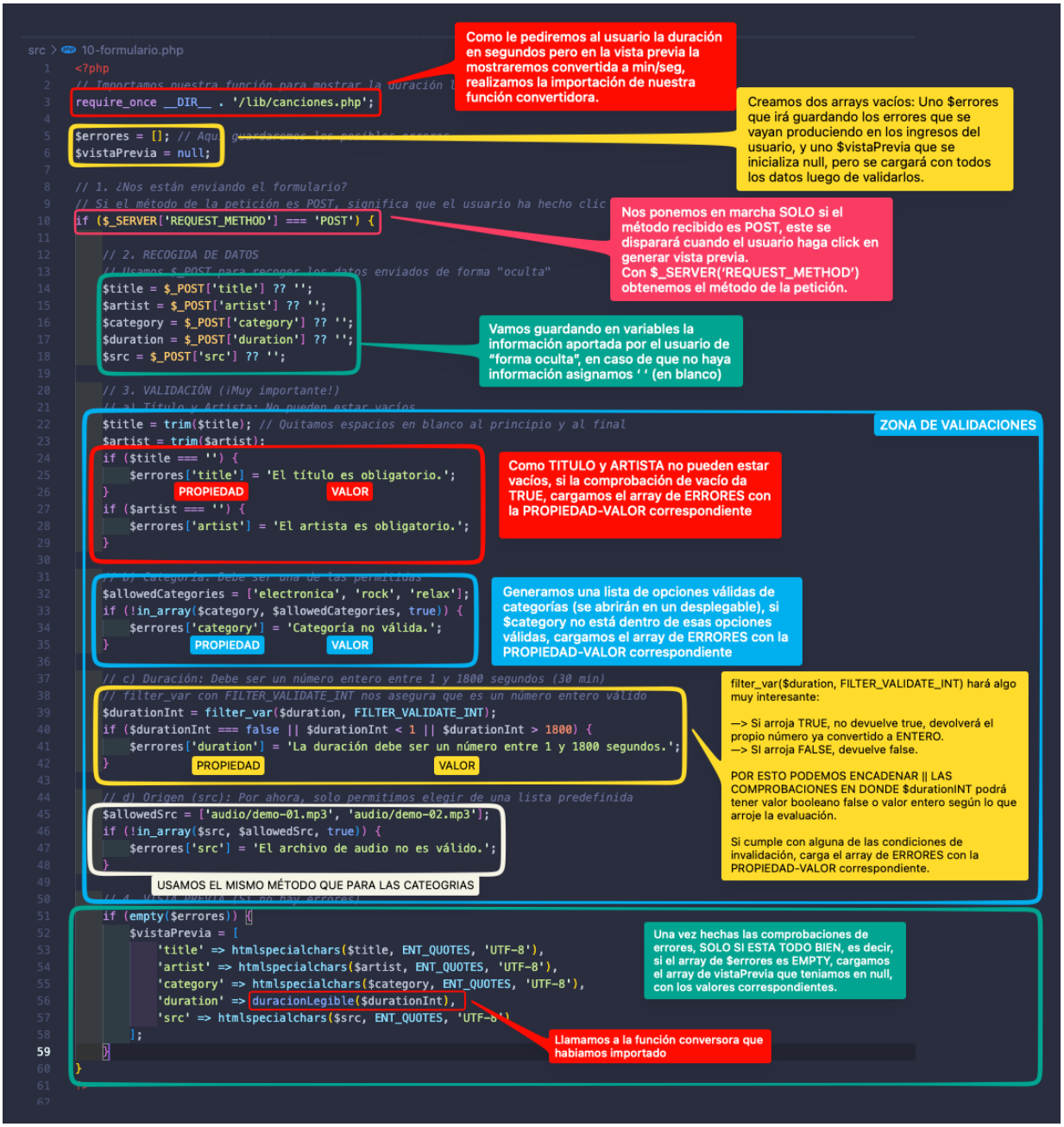
Si un hacker intentara meter codigo malicioso o ingresar una categoría inventada se encuentra con nuestra barrera de seguridad.
Como la categoria no está en nuestra lista de permitidos, se devuelve el error 400 y se detiene la pagina.



7) PREPARANDO EL FORMULARIO PARA AÑADIR UNA CANCIÓN

En este punto, crearemos un “formulario” en el cual el usuario (administrador) introduce los datos de una nueva canción.

No haremos un guardado directo al catálogo, sino que crearemos una “vista previa” para asegurarnos de que la información llega y está correctamente validada antes de pasar al catálogo.



VEAMOS LA DINÁMICA DEL HTML QUE TAMBIÉN ES BASTANTE INTERESANTE:



Entendiendo para que sirve esto:

```
<input type="text" id="artist" name="artist" value="<?= htmlspecialchars($_POST['artist'] ?? '', ENT_QUOTES) ?>">
```

Esto se denomina Operador de Fusión NUll (Null Coalescing Operator) y nos permite evitar errores en la página cuando se entra por primera vez.

Cuando entramos a la página por primera vez y aún no se han enviado datos, $_POST aún no existe, no tiene nada en su interior.

Si nosotros tenemos establecido este condicional para recuperar los datos ya ingresados por el usuario y esos datos aún no existen, la página intentará ejecutar esa condición para evaluar un resultado y explotará.

Por ello se establece un salvavidas:

En lenguaje humano, `$_POST['category'] ?? ''` significa exactamente esto:

> *"Intenta sacar el valor de* `$_POST['category']`. **Pero si no existe** *(porque es la primera vez que entramos a la página), no te asustes, usa simplemente* **un texto vacío** `''` *como Plan B".*

Con esto, la ejecución quedará asi:

Escenario 1: Es la primera vez que entramos a la página

1. PHP intenta leer `$_POST['category']`. ¡No existe!
2. El salvavidas `?? ''` entra en acción y le da a PHP un texto vacío `''`.
3. PHP se pregunta: ¿Es `'' === 'rock'` ? **Falso.**
4. El operador ternario (el `? :` general) dice: Como es falso, imprimo vacío `''`. La opción no se marca.

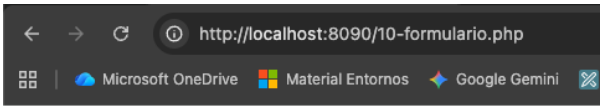
Escenario 2: El usuario eligió "Relax" y dio a Enviar (y hubo un error en otro campo)

1. PHP intenta leer `$_POST['category']`. ¡Sí existe! Vale `'relax'`.
2. Como sí existe, el salvavidas `?? ''` es ignorado.
3. PHP se pregunta: ¿Es `'relax' === 'rock'` ? **Falso.**
4. Como es falso, imprime vacío `''`. La opción rock no se marca.

Escenario 3: El usuario eligió "Rock" y dio a Enviar (y hubo un error en otro campo)

1. PHP intenta leer `$_POST['category']`. ¡Sí existe! Vale `'rock'`.
2. El salvavidas `?? ''` es ignorado.
3. PHP se pregunta: ¿Es `'rock' === 'rock'` ? **¡Verdadero!**
4. Como es verdadero, imprime `'selected'`. El HTML final queda `<option value="rock" selected>Rock</option>` **y la opción se queda marcada en pantalla para el usuario.**

PRUEBAS EN EL FORMULARIO:



Añadir Nueva Canción

Título:

Artista:

Categoría: Selecciona...

Duración (segundos):

Archivo de audio: Selecciona...

Generar Vista Previa

PRIMERA CARGA

Añadir Nueva Canción

Título: llamada de emergencia

Artista:

omitimos el artista y enviamos

Categoría: Electrónica

Duración (segundos): 120

Archivo de audio: Demo 01

Generar Vista Previa

Añadir Nueva Canción

Título: llamada de emergencia

Artista: El artista es obligatorio.

Categoría: Electrónica

Duración (segundos): 120

La página se recarga a sí misma, imprime el codigo de error pero recupera todo lo que el usuario ya habia escrito gracias a nuestro condicional.

Archivo de audio: Demo 01

Generar Vista Previa

Añadir Nueva Canción

Vista previa, canción no guardada

- **Título:** llamada de emergencia
- **Artista:** daddy yankee
- **Categoría:** electronica
- **Duración:** 2 minutos y 0 segundos
- **Archivo:** audio/demo-01.mp3

Título: llamada de emergencia

Artista: daddy yankee

TODOS LOS CAMBIOS SUPERARON EL CONTROL DE VALIDACIÓN, $ERRORS está vacío, entonces vistaPrevia se carga correctamente en la pantalla.

Categoría: Electrónica

Duración (segundos): 120

Archivo de audio: Demo 01

Generar Vista Previa

8) SEPARACIÓN DE CÓDIGO PÚBLICO DE DATOS PRIVADOS Y PREPARACION DE BBDD

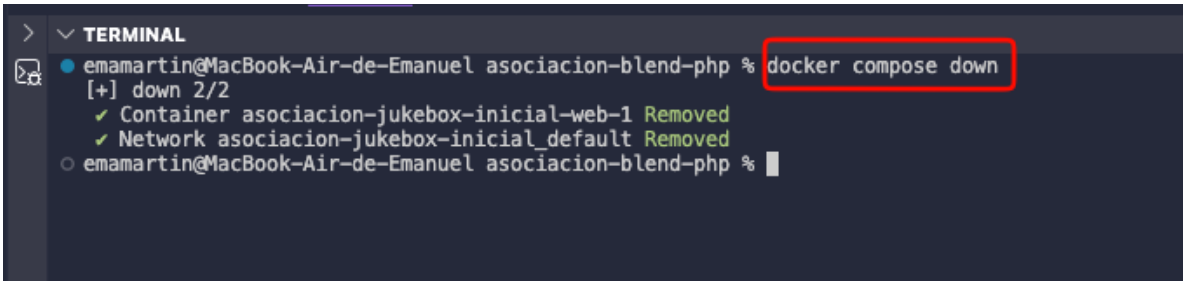
Hasta ahora, hemos tenido nuestro catálogo de canciones (el array) mezclado con el código público. Es decir, si en la url ingresábamos 07-catalogo.php cualquier persona podía acceder a él.

En un proyecto real, esto es un riesgo de seguridad enorme. Además, solo hemos estado utilizando un contenedor básico (PHP y Apache).

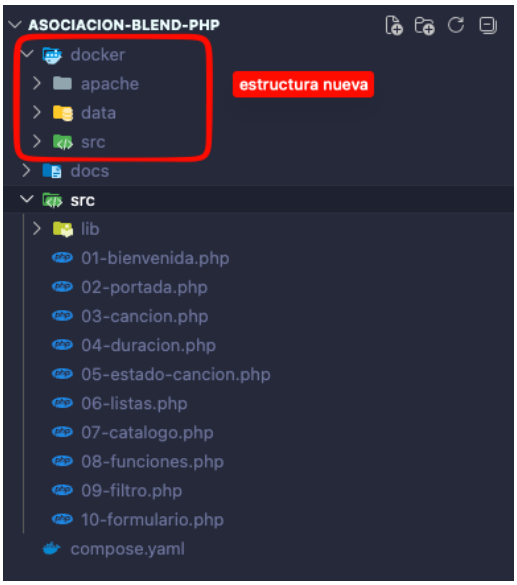
Lo que haremos será crear una arquitectura multicontenedor más profesional.

Para ello, crearemos una copia paralela del proyecto y la haremos correr en el puerto 8091.

A) Detenemos el servidor



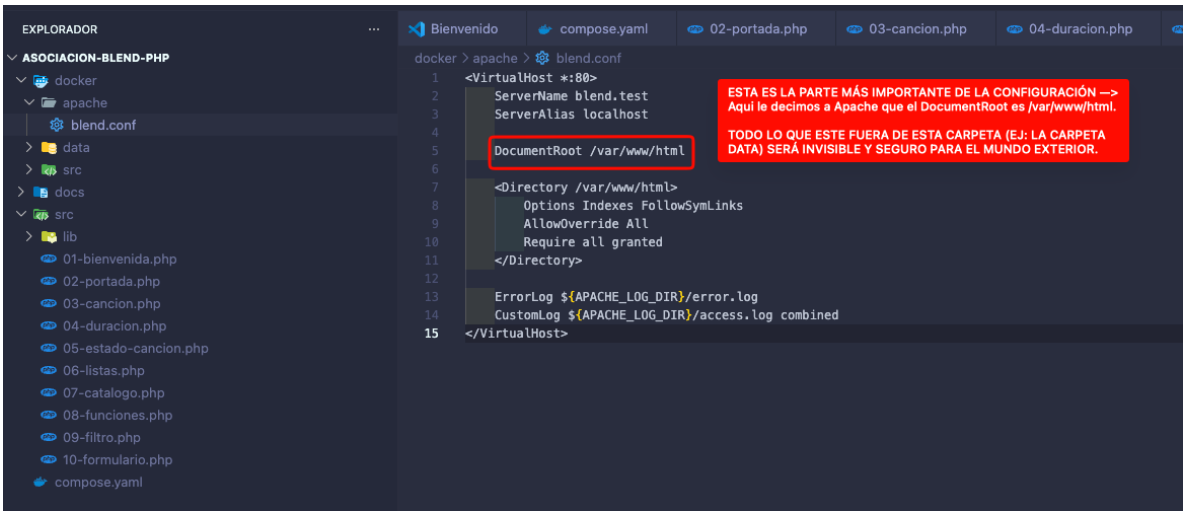
B) Crearemos una carpeta llamada docker en la raíz del proyecto y en ella pegaremos una copia de toda la carpeta src y lib. Además dentro de la carpeta docker, crearemos dos carpetas vacías: apache y data.



C) Crearemos el archivo de configuración Apache

Tenemos que decirle a Apache (el encargado de gestionar las peticiones) cómo debe servir las páginas en este nuevo entorno.

Para ello creamos el archivo blend.conf con la siguiente estructura:

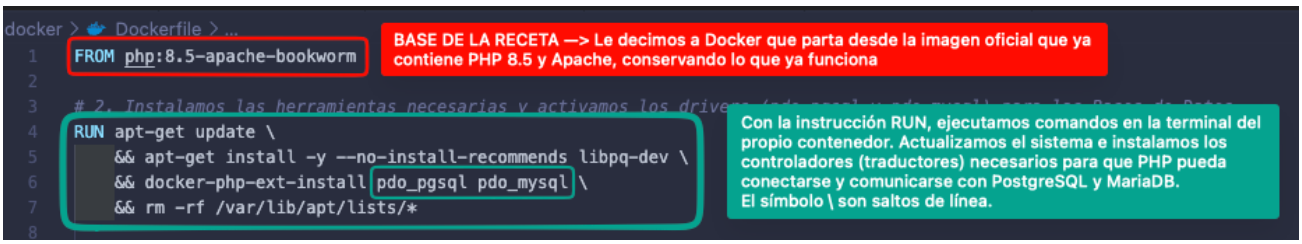


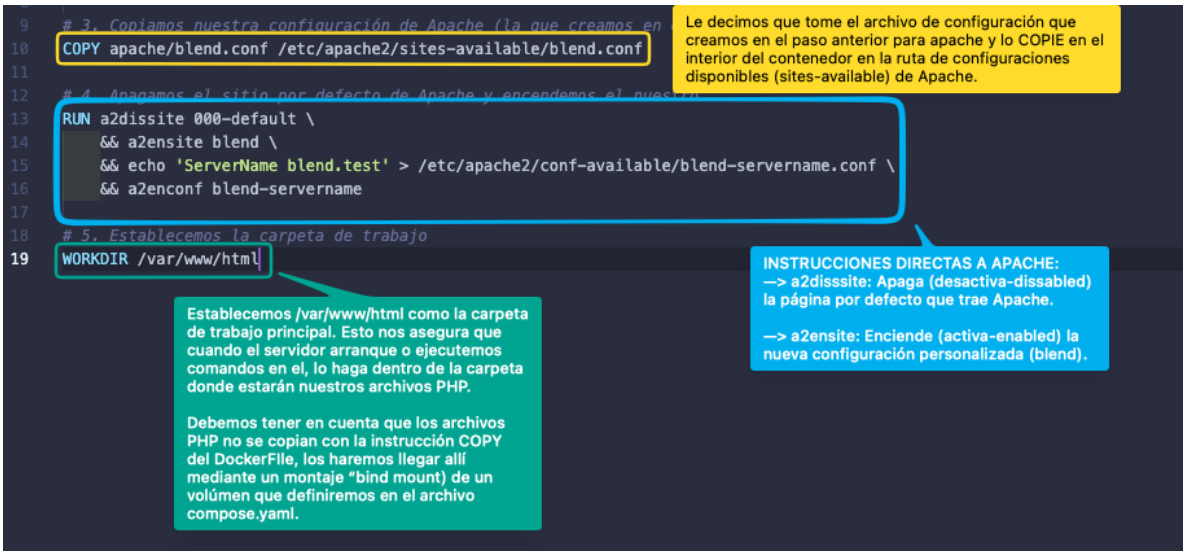
D) PREPARACIÓN DEL DOCKERFILE

El **Dockerfile** actúa como un libro de recetas o manual de instrucciones que nos permite construir nuestro servidor web a medida ya que la imagen prefabricada que veníamos usando, no tiene todo lo que necesitaremos para esta nueva etapa del proyecto.

Hasta el momento, veníamos utilizando la imagen oficial php:8.5-apache-bookworm, que es como comprar un ordenador que ya trae PHP y Apache instalados de fábrica.

Sin embargo, a medida que la aplicación crece, necesitamos que PHP se conecte a dos bases de datos distintas: PostgreSQL y MariaDB. Como esa imagen básica no tiene los controladores (“traductores”) para hablar con esas bases de datos, utilizamos Dockerfile para añadir lo que falta, tomando como base esa imagen original. LA COMPLETAMOS.





E) EL DIRECTOR DE LA ORQUESTA → COMPOSE.YAML

Antes de la creación de este archivo, nos detendremos un momento a hablar de un tema importante: Las contraseñas.

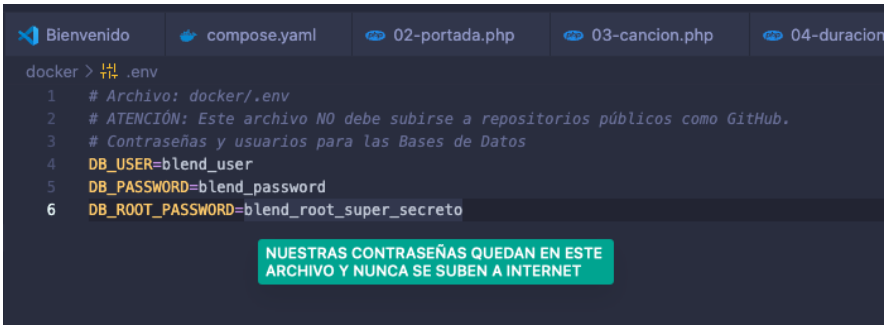
Como en el compose.yaml ponemos información referente a la BBDD (usuario y contraseña), ponerlo directamente en este archivo (hardcodear contraseñas) es una práctica de seguridad terrible.

¿Por qué? → Basicamente porque si subimos nuestro proyecto, incluyendo este archivo, a GitHub, cualquier persona del mundo podría ver las claves de nuestra BBDD.

La solución a esto de forma profesional es crear un **archivo de variables de entorno (.env).**

Este archivo .env se queda en nuestro ordenador y nunca se sube a internet, y el compose.yaml leerá las contraseñas desde ahí.

Para ello, dentro de la carpeta docker, al mismo nivel que el dockerfile, crearemos el archivo .env.



Además de esto, como el archivo .env no se sube a internet, ¿Cómo sabe otro programador de nuestro equipo que variables necesita configurar en su ordenador para arrancar el proyecto?

- Para esto, se crea una plantilla con valores falsos, que si se comparte en GitHub. Esta plantilla se nombra .env.example y se ubica en la misma raíz.



CREACIÓN DEL ARCHIVO COMPOSE.YAML

Para entender el funcionamiento del compose.yaml vamos a darnos contexto.

El Dockerfile es **el manual de instrucciones** para construir una máquina específica (nuestro servidor web modificado/estilizado a nuestra manera).

Pero Docker files eso, un manual, no hace nada por sí solo. Necesitamos a alguien que lea esae manual, construya la máquina, construya también las BBDD y conecte todo entre sí.

Es como un director de orquesta, y ese es el **compose.yaml.**

Si bien en la etapa 1 ya habíamos creado un compose.yaml, en esa etapa era muy básico ya que solo levantábamos un contenedor (el servidor web).

Ahora nuestro compose.yaml tendrá tres misiones:

- Orquestar múltiples contenedores (Servicios)

En lugar de levantar una sola cosa, le ordenará a Docker que encienda tres máquinas independientes al mismo tiempo.

- Servicio web → Le dirá que no use una imagen prefabricada, sino que lea nuestro Dockerfile para constuir el servidor a medida en el puerto 8091.
- Servicio postgres → Descargará el motor de BBDD PostgreSQL.
- Servicio MariaDB → Descargará el motor de BBDD MariaDB.

- **Crear una red privada inteligente**

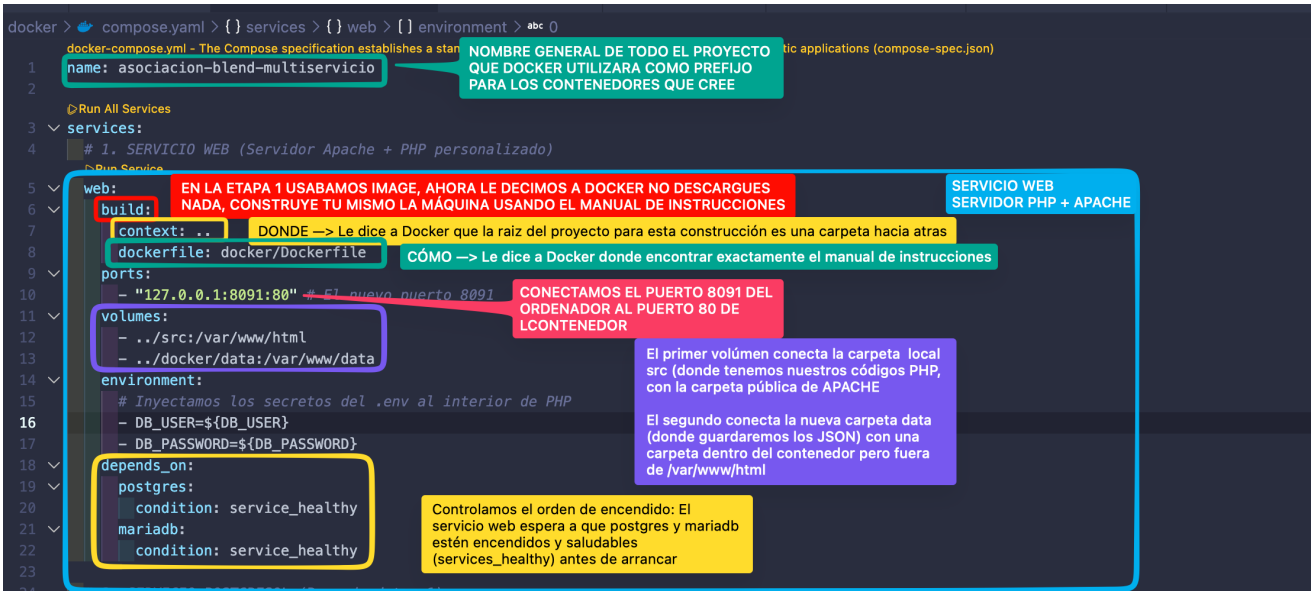
Al levantar tres contenedores desde el mismo archivo, Docker crea automáticamente una red wifi invisible entre ellos. Nuestro código PHP no necesitará saber la IP de la BBDD. Para hablar con POsgreSQL, PHP simplemente llamará a la puerta del host llamado postgres.
Para hablar con MariaDB, llamará al host mariadb.

- **Gestionar la seguridad (.env) y la Persistencia (volúmenes)**

- Actúa como puente → El compose.yaml será el encargado de mirar el archivo local .env, aspirar las contraseñas que le hemos escrito ahí, e inyectarlas de forma segura en las BBDD usando la variable ${DB_PASSWORD} sin que queden escritas en el código.
- Gestiona los discos duros (volúmenes) → Conecta nuestra carpeta local src/ al contenedor web para que podamos seguir editando código en tiempo real. Pero además, creará volúmenes especiales para postgres y mariadb.
  Si apagamos Docker, esos volúmenes guardan los datos a salvo para que no perdamos tablas ni usuarios.
- Estableceremos ordenes de encendido → depends_on y healtcheck serán fundamentales. Le diremos a compose.yaml que la web no debe arrancar a lo loco, sino que primero debe encender las BBDD y hacerles un chequeo médico (healthcheck) para verificar primero si están listas para recibir conexiones. Una vez que lo tenga, encenderá el servidor web.

ESTRUCTURA DE NUESTRO DOCKERFILE

Primer servicio → El servidor web



NOTA: HAY UN ERROR EN LOS VOLUMENES DEL SERVICIO WEB, DEBEN QUEDAR ASÍ:

```
    - "127.0.0.1:8091:80" # El nuevo puerto 8091
  volumes:
    - ./src:/var/www/html
    - ./data:/var/www/data
```

Segundo servicio → PostgreSQL

```
# 2. SERVICIO POSTGRESQL (Base de datos 1)
▷Run Service
postgres:
  image: postgres:17-bookworm
  environment:
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: blend_pg
  volumes:
    - pg_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d blend_pg"]
    interval: 5s
    timeout: 5s
    retries: 5

# 3. SERVICIO MARIADB (Base de datos 2)
```

PARA LA BBDD SI USAMOS UNA IMAGEN PREFABIRCADA OFICIAL

Le inyectamos las variables de entorno que PostgreSQL necesita para configurarse la primera vez que arranca, leyendo la información del archivo .env para evitar que las contraseñas queden expuestas en el código

NOMBRE DE LA BBDD QUE CREARÁ AL INICIAR

Aquí creamos un volúmen de datos con persistencia, un volumen docker real para que los datos no se borren cuando apaguemos el contenedor

CON EL HEALTCHECK, CADA 5 SEGUNDOS (INTERVAL) EJECUTA EL COMANDO PG_ISREADY... PARA PREGUNTARLE A POSTGRESQL SI ESTÁ LISTO PARA RECIBIR CONEXIONES. SOLO CUANDO REASPONDE SI PASA A ESTADO HEALTHY

Tercer servicio → MariaDB

```
# 3. SERVICIO MARIADB (Base de datos 2)
▷Run Service
mariadb:
  image: mariadb:11.4
  environment:
    MARIADB_USER: ${DB_USER}
    MARIADB_PASSWORD: ${DB_PASSWORD}
    MARIADB_DATABASE: blend_maria
    MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
  volumes:
    - maria_data:/var/lib/mysql
  healthcheck:
    test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
    interval: 5s
    timeout: 5s
    retries: 5
```

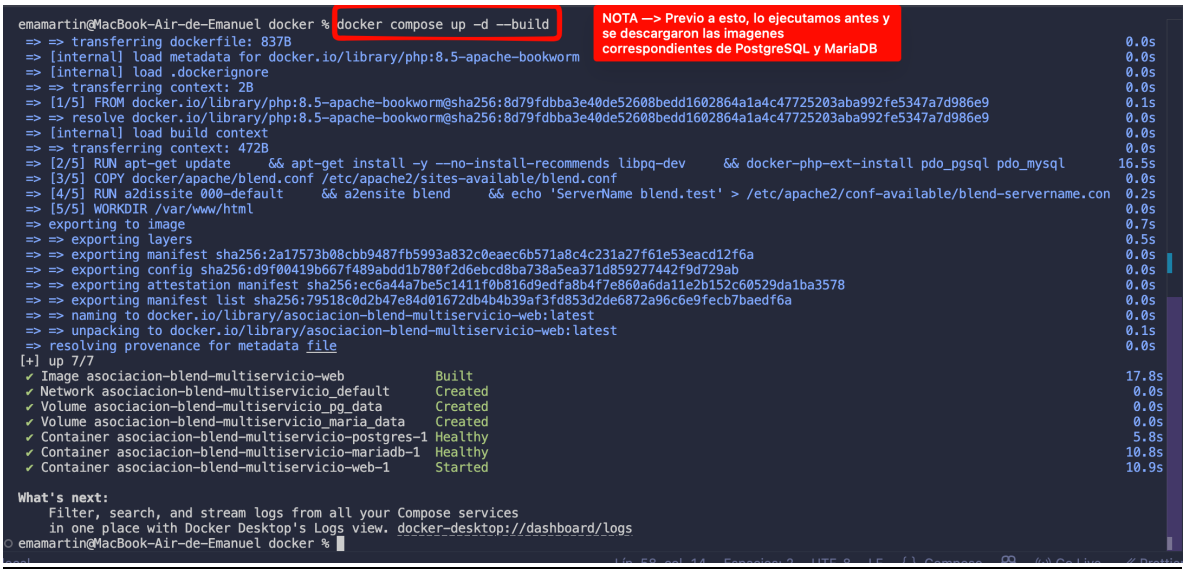
SE REPITE LO MISMO QUE CON POSTGRESQL

```
volumes:
  pg_data:
  maria_data:
```

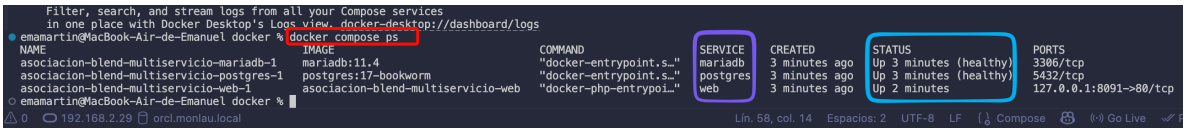
Al final del código definimos los volúmenes perisistentes que coinciden con las rutas que especificamos antes para las BBDD

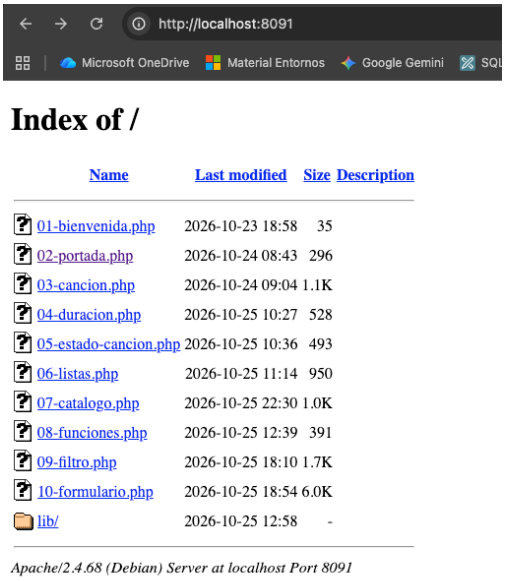
ENCENDEMOS TODA LA NUEVA ESTRUCTURA

Antes de ejecutar el comando, es fundamental estar situados en la carpeta docker.



Si ejecutamos Docker compose ps podemos comprobar si efectivamente se han creado los tres servicios y si los tres funcionan correctamente.



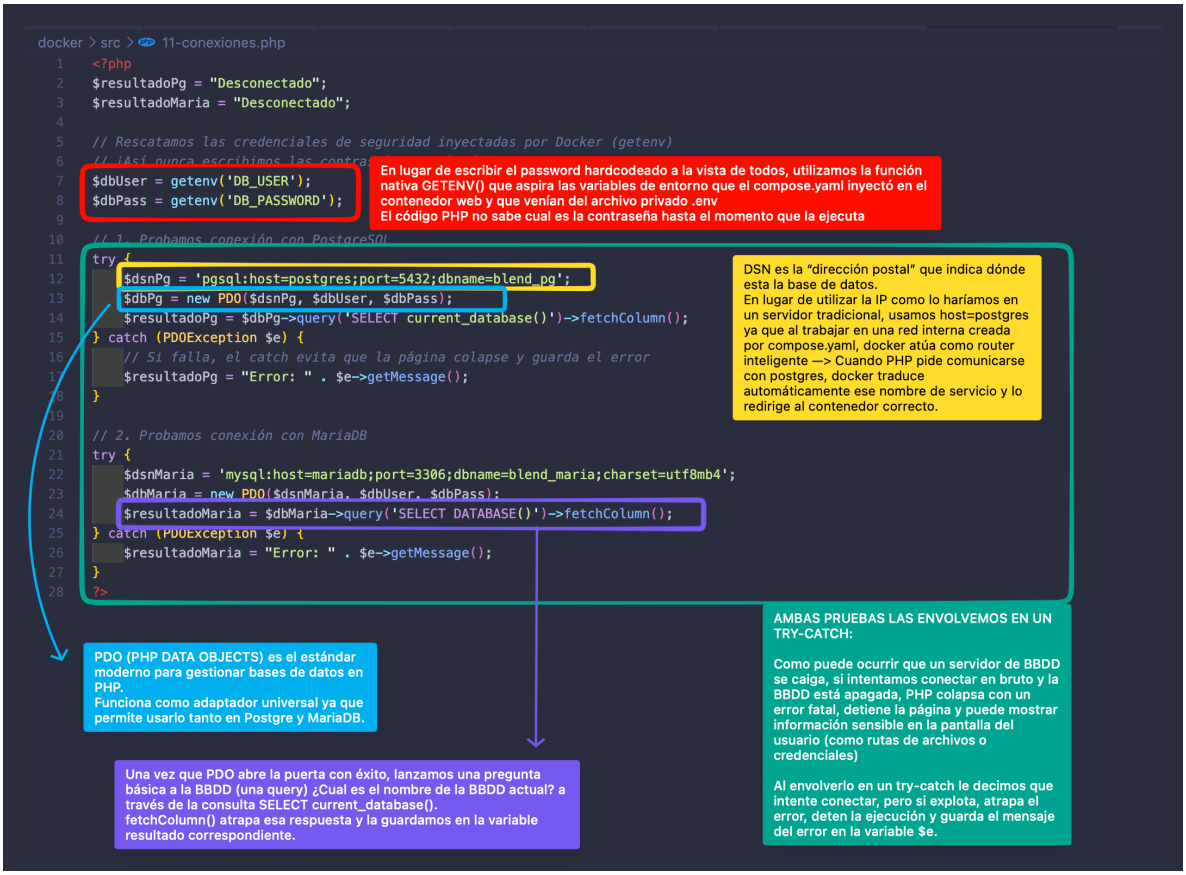


Como en DockerfIle establecimos conexión entre la carpeta local src con la carpeta publica de apache var/www/html, al entrar en la página visualizamos todos los archivos.

```
emamartin@MacBook-Air-de-Emanuel docker % curl -i http://localhost:8091/
HTTP/1.1 200 OK
Date: Tue, 27 Oct 2026 17:46:23 GMT
Server: Apache/2.4.68 (Debian)
Vary: Accept-Encoding
Content-Length: 2896
Content-Type: text/html;charset=UTF-8
```

F) COMPROBACION DE CONEXIÓN A LAS BASES DE DATOS

Para demostrar que todo funciona correctamente, crearemos el siguiente archivo 11-conexiones.php



G) EL ARCHIVO .GITIGNORE Y EL ARCHIVO .DOCKERIGNORE

Cuando trabajamos en un proyecto, solemos usar herramientas de control de versiones como GIT (para guardar el historial de código y subirlo a GitHub) y Docker (para construir los contenedores).

El peligro radica en que GIT rastrea y guarda todos los archivos de la carpeta por defecto. Si subimos el proyecto a un repositorio, subiríamos también nuestro archivo **docker/.env**

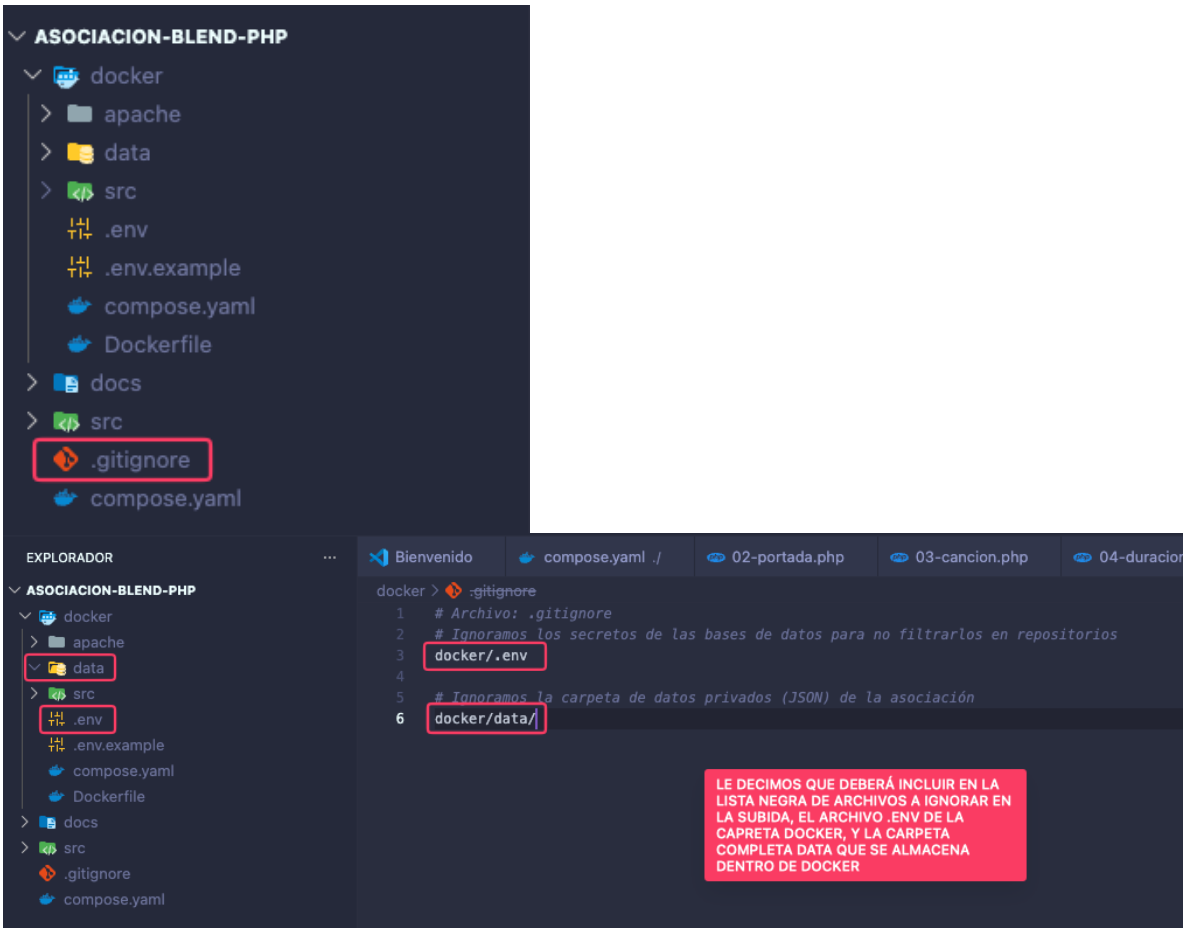
revelando así las contraseñas reales de nuestra BBDD y si la carpeta **docker/data/** donde revelaríamos datos privados de los usuarios y canciones de la asociación.

Para evitar esta brecha de seguridad, usamos archivos de exclusión.

CREACIÓN DEL .GITIGNORE (ESCUDO PARA GIT)

El archivo .gitignore funciona como una **lista negra** en la cual TODO lo que escribamos dentro de él será completamente ignorado por GIT, como si esos archivos fueran invisibles.

Para ello crearemos el archivo en la raíz de nuestro proyecto:



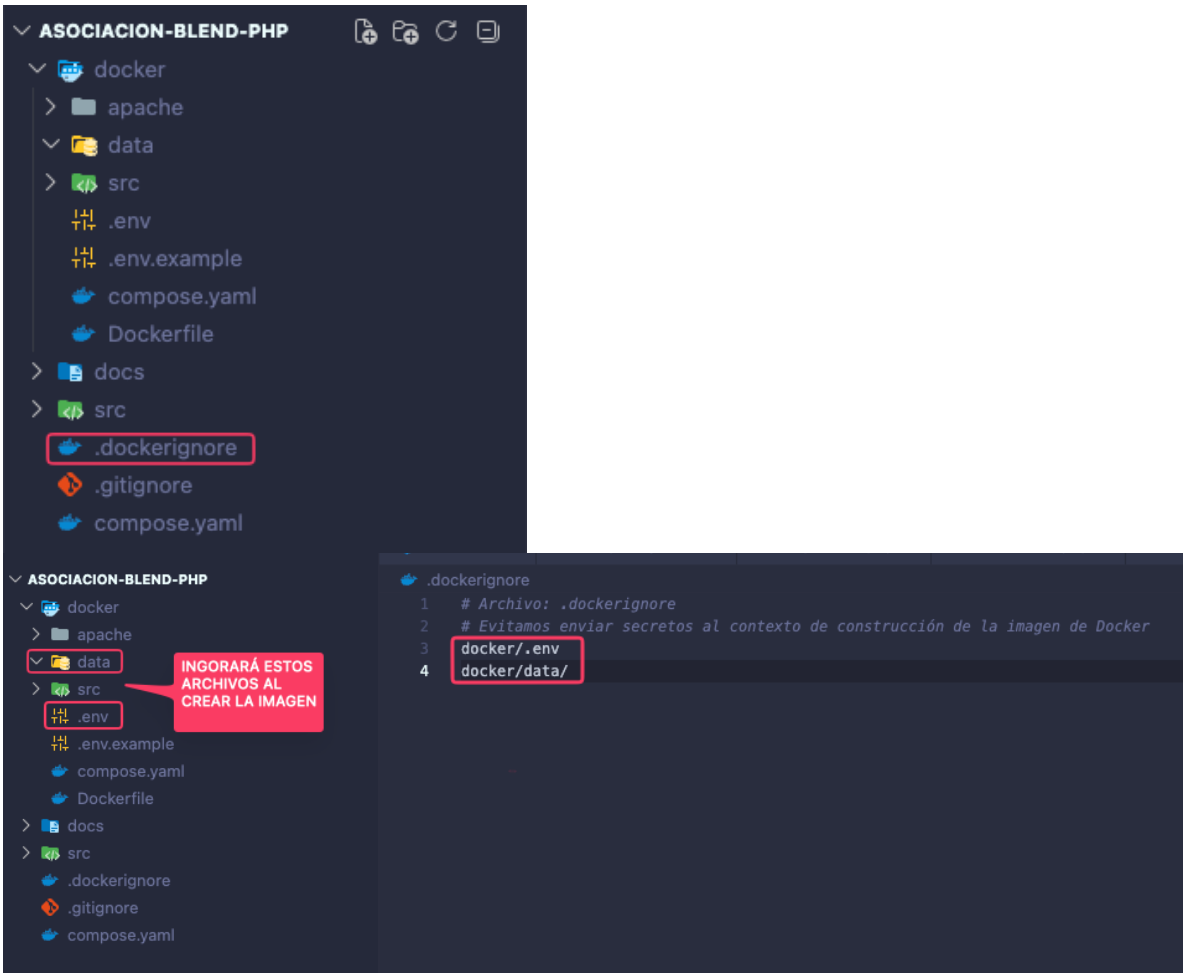
De esta manera esta carpeta y archivo quedarán excluídas de la subida, y respecto a los .env, solo se subirá el de ejemplo (.env.example)

CREACIÓN DEL .DOCKERIGNORE (ESCUDO PARA DOCKER)

De la misma manera que GIT copia todo, cuando ejecutamos el comando **docker compose up –build,** Docker toma TODA la carpeta del proyecto (su build context) y la envía a su motor interno para construir la imagen del servidor.

Como no queremos que nuestras contraseñas reales ni nuestros datos privados se queden incrustados accidentalmente dentro de la imagen del servidor Apache que estamos construyendo, creamos un archivo **.dockerignore.**

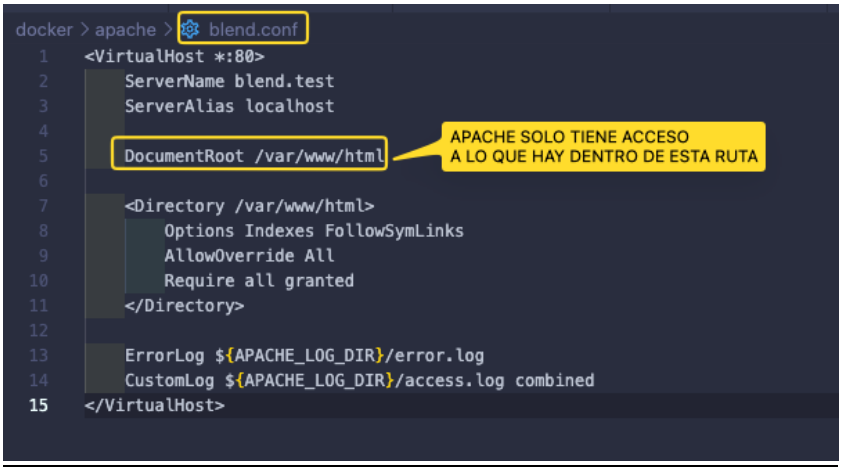
Al igual que antes, en la raíz absoluta, crearemos el nuevo archivo con el mismo contenido.

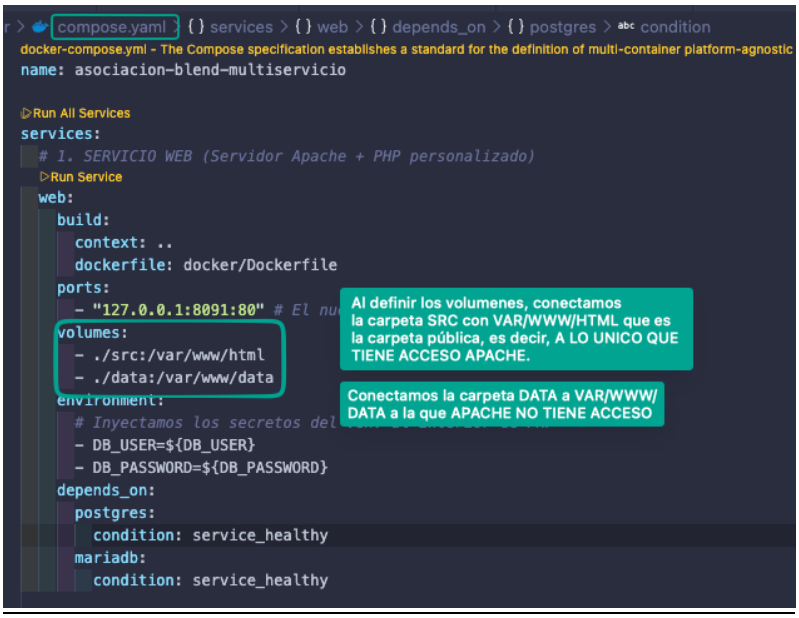


OTRA CONFIGURACIÓN DE SEGURIDAD: LAS CARPETAS PÚBLICAS Y PRIVADAS

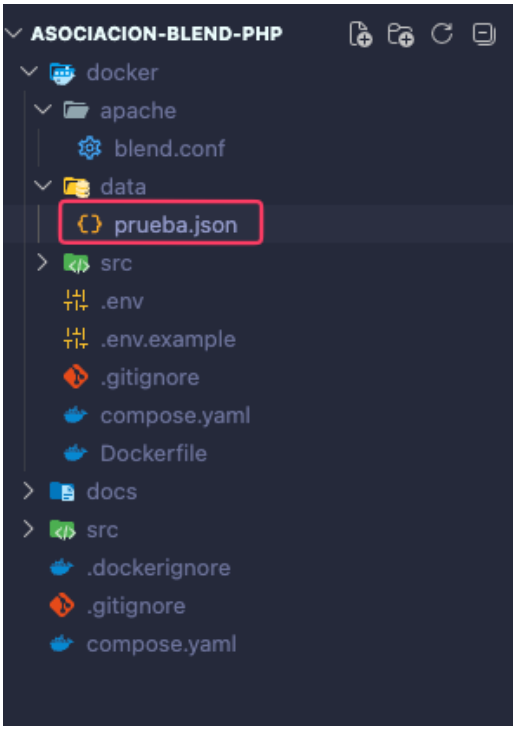
Antes establecimos que el archivo .gitignore y .dockerignore protegerán nuestro código fuente, evitando que secretos (contraseñas BBDD) y datos privados (datos de usuarios y canciones) viajen accidentalemnte a GITHUB o se incrusten en la imagen de Docker. **ESTO ES UNA PROTECCIÓN EN FASE DE DESARROLLO/DESPLIEGUE.**

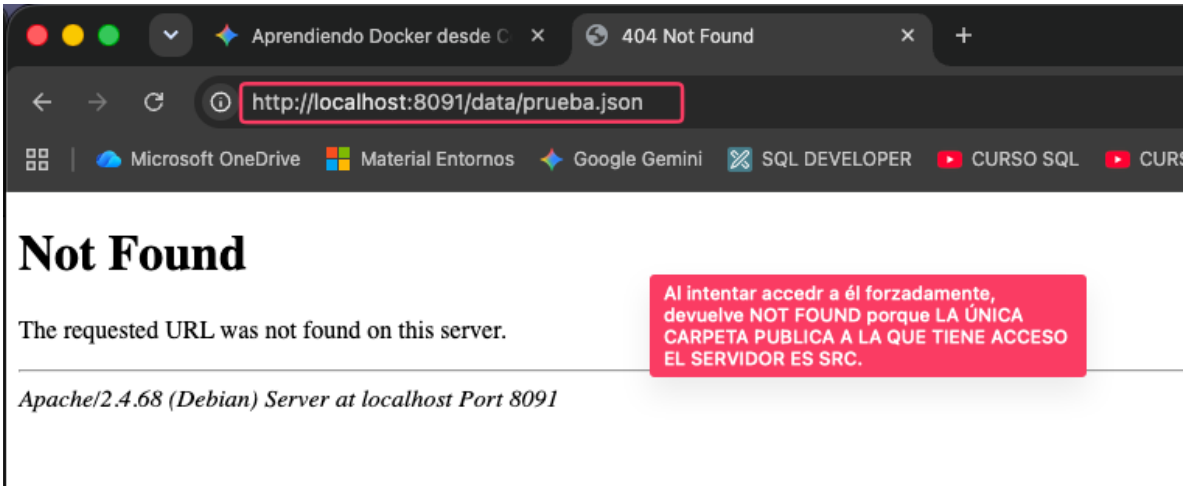
Sin embargo antes, al realizar la configuración de nuestro archivo blend.conf determinamos que el Document Root (carpeta pública a la que tiene acceso Apache) fuera la carpeta /var/www/html y en el compose.yaml pusimos conectamos la carpeta data a var/www/data, dejándola fuera del alcance de Apache. ESTO ES UNA PROTECCIÓN EN FASE DE EJECUCIÓN, CUANDO LA WEB ESTÁ ENCENDIDA.





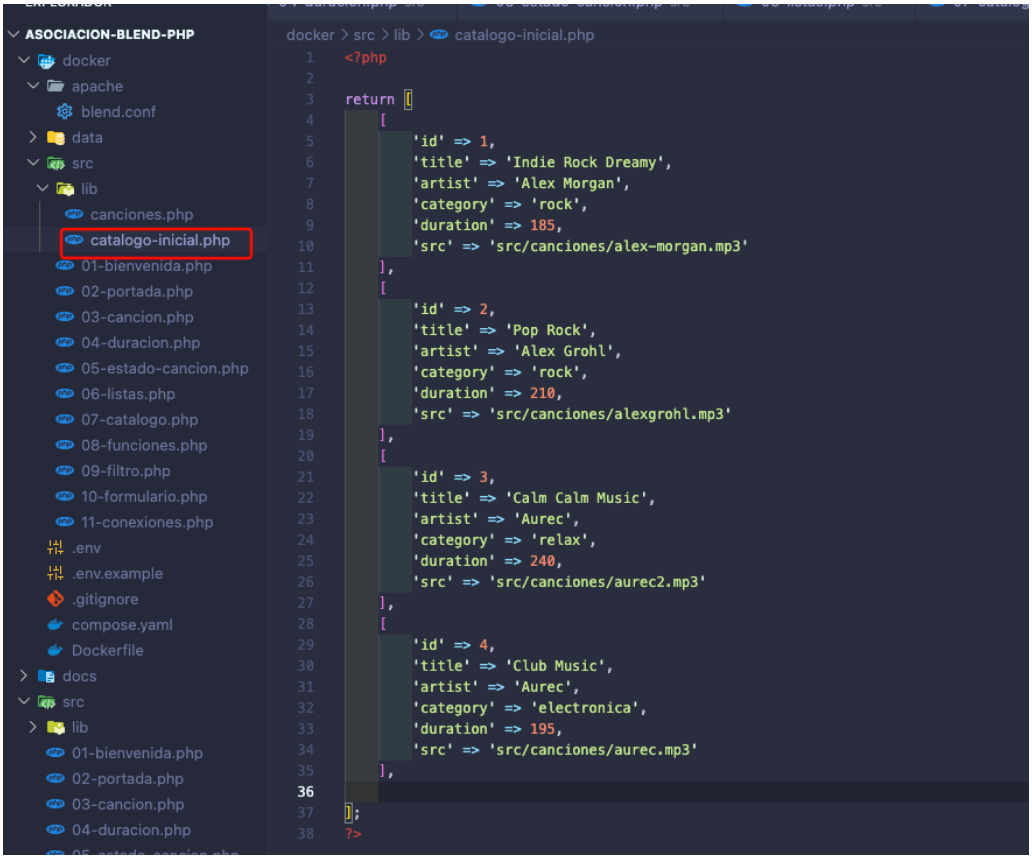
Podemos realizar una prueba de esta capa de seguridad en ejecución, si creamos un archivo dentro de la carpeta data e intentamos acceder a él a través del navegador:





9) LECTURA DE DATOS DE FORMA SEGURA

Hasta ahora, hemos tenido el catálogo musical atrapado dentro del archivo PHP llamado catálogo-inicial.php.



Sin embargo esto no es algo útil en la práctica real, ya que no suelen mezclarse los datos (el catálogo) con el motor lógico PHP, sino que se separan.

Pensemos en PHP como el cocinero y en los datos como los ingredientes. Lo que haremos ahora será sacar los ingredientes de la cocina y guardarlos en una despensa segura (un archivo .json privado).

A partir de ahora, el cocinero PHP, tendrá que ir a la despensa a buscar los ingredientes cada vez que alguien le pida el menú.

9.1 CREANDO LA DESPENSA (CANCIONES.JSON)

Mudaremos el catálogo a un archivo llamado canciones.json, que guardaremos en la ruta docker/data/canciones.json.

Tal como vimos antes cuando realizamos la prueba segura de acceso, el servidor no tendrá acceso directo a este archivo, por lo que también es una medida de seguridad.




```json
[
    {
        "id": 1,
        "title": "Indie Rock Dreamy",
        "artist": "Alex Morgan",
        "category": "rock",
        "duration": 185,
        "src": "src/canciones/alex-morgan.mp3"
    },
    {
        "id": 2,
        "title": "Pop Rock",
        "artist": "Alex Grohl",
        "category": "rock",
        "duration": 210,
        "src": "src/canciones/alexgrohl.mp3"
    },
    {
        "id": 3,
        "title": "Calm Calm Music",
        "artist": "Aurec",
        "category": "relax",
        "duration": 240,
        "src": "src/canciones/aurec2.mp3"
    },
    {
        "id": 4,
        "title": "Club Music",
        "artist": "Aurec",
        "category": "electronica",
        "duration": 195,
        "src": "src/canciones/aurec.mp3"
    }

]
```

Al guardar los datos en un archivo .json en otro directorio, ahora los archivos de la web pública (como lo hacia catalogo.php) ya no pueden ir a buscar directamente las canciones como si lo hacían con catalogo-inicial.php.

Ahora necesitan un **intermediario** → Será una función especializada en leer archivos .json, que lo convertirá en un Array que PHP pueda entender.

9.2 CREAMOS LA FUNCION TRADUCTORA EN docker/src/lib.



9.3 CREAMOS EL NUEVO CATÁLOGO PÚBLICO

Antes teníamos el archivo 07-catalogo.php que hacia esto:



Ahora crearemos un nuevo archivo 12-catalogo-json.php que hará lo siguiente:

- Pedirá los datos al intermediario (almacen-json.php) que como vimos antes, buscará en el catálogo privado (canciones.json), y le retornará el array listo para que simplemente lo recorra y lo pinte.



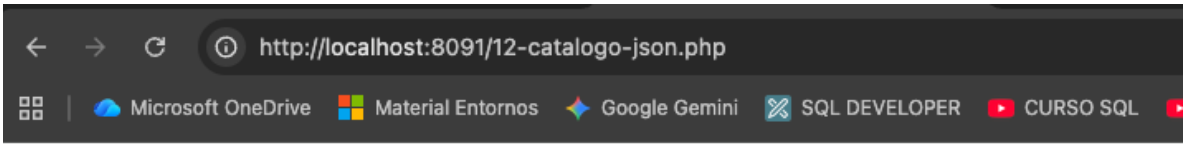

```php
<?php
// Traemos nuestras herramientas de formato (para que d
require_once __DIR__ . '/lib/canciones.php';

// Le pedimos al "chico de los recados" que nos traiga
$songs = require __DIR__ . '/lib/almacen-json.php';
?>

<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Catálogo Oficial JSON</title>
</head>
<body>
    <h1>Catálogo de Canciones Blend (Desde JSON)</h1>

    <!-- SI LA BANDEJA LLEGÓ VACÍA -->
    <?php if (empty($songs)): ?>
        <p><strong>Todavía no hay canciones o hubo un problema técnico en la despensa.</strong></p>
    <!-- SI LA BANDEJA LLEGÓ LLENA -->
    <?php else: ?>
        <ul>
            <?php foreach ($songs as $song): ?>
                <li>
                    <strong><?= htmlspecialchars($song['title'], ENT_QUOTES, 'UTF-8') ?></strong>
                    por <?= htmlspecialchars($song['artist'], ENT_QUOTES, 'UTF-8') ?>
                    (<?= duracionLegible($song['duration']) ?>)
                </li>
            <?php endforeach; ?>
        </ul>
    <?php endif; ?>
</body>
</html>
```

REALIZAMOS LA PRUEBA PARA COMPROBAR SU FUNCIONAMIENTO → Al acceder a localhost:8091/12-catálogo-json.php, deberíamos ver el nuevo catálogo que creamos



Catálogo de Canciones Blend (Desde JSON)

- **Indie Rock Dreamy** por Alex Morgan (3 minutos y 5 segundos)
- **Pop Rock** por Alex Grohl (3 minutos y 30 segundos)
- **Calm Calm Music** por Aurec (4 minutos y 0 segundos)
- **Club Music** por Aurec (3 minutos y 15 segundos)

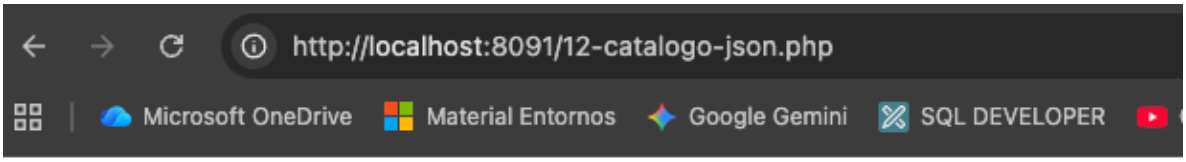
Para probar si el try-catch que hemos previsto para los errores funciona bien, hacemos una modificación en el archivo raíz canciones.json para forzar el error:

docker > data > {} canciones.json > { } 2

```
[
    {
        "id": 1,
        "title": "Indie Rock Dreamy",
        "artist": "Alex Morgan",
        "category": "rock",
        "duration": 185,
        "src": "src/canciones/alex-morgan.mp3"
    }
    {
        "id": 2,
        "title": "Pop Rock",
        "artist": "Alex Grohl",
        "category": "rock",
        "duration": 210,
        "src": "src/canciones/alexgrohl.mp3"
    },
    {
        "id": 3,
        "title": "Calm Calm Music",
        "artist": "Aurec",
        "category": "relax",
        "duration": 240,
        "src": "src/canciones/aurec2.mp3"
    },
    {
        "id": 4,
        "title": "Club Music",
        "artist": "Aurec",
        "category": "electronica",
        "duration": 195,
        "src": "src/canciones/aurec.mp3"
    }

]
```

quitamos la coma que rompe la estructura



Error crítico: El archivo JSON está dañado. Revisa la sintaxis.

Catálogo de Canciones Blend (Desde JSON)

Todavía no hay canciones o hubo un problema técnico en la despensa.

El try/catch funciona capturando el error y mostrando un mensaje en lugar de colapsar la página.

10) CREANDO UNA VERDADERA APLICACIÓN WEB

En este bloque le enseñaremos a nuestro servidor a recordar quiénes somos (sesiones) ya protegernos de ataques de suplantación (CSRF).

10.1 PREPARAR LA MEMORIA DEL SERVIDOR (SESION)

Para entender el funcionamiento de esta etapa, haremos una analogía:

EL CLUB VIP Y EL SELLO DE CERA

Por defecto, la web tiene "amnesia" (El protocolo HTTP no tiene estado).
Cada vez que hacemos click en un enlace o enviamos un formulario, el servidor nos trata como un completo desconocido.

- La sesión ($_SESSION): Es como la lista VIP en la puerta de una discoteca. El servidor anota ahí nuestros datos (ej: "Este usuario ya puso su contraseña").

- La cookie (JUKEBOXSESSID): Para que no tengamos que decir nuestro nombre en cada puerta de la discoteca, el servidor nos pone una "pulsera de papel" (la cookie) en el navegador.
  En cada click, el navegador enseña la pulsera y el servidor dice "te recuerdo, estás en la lista.

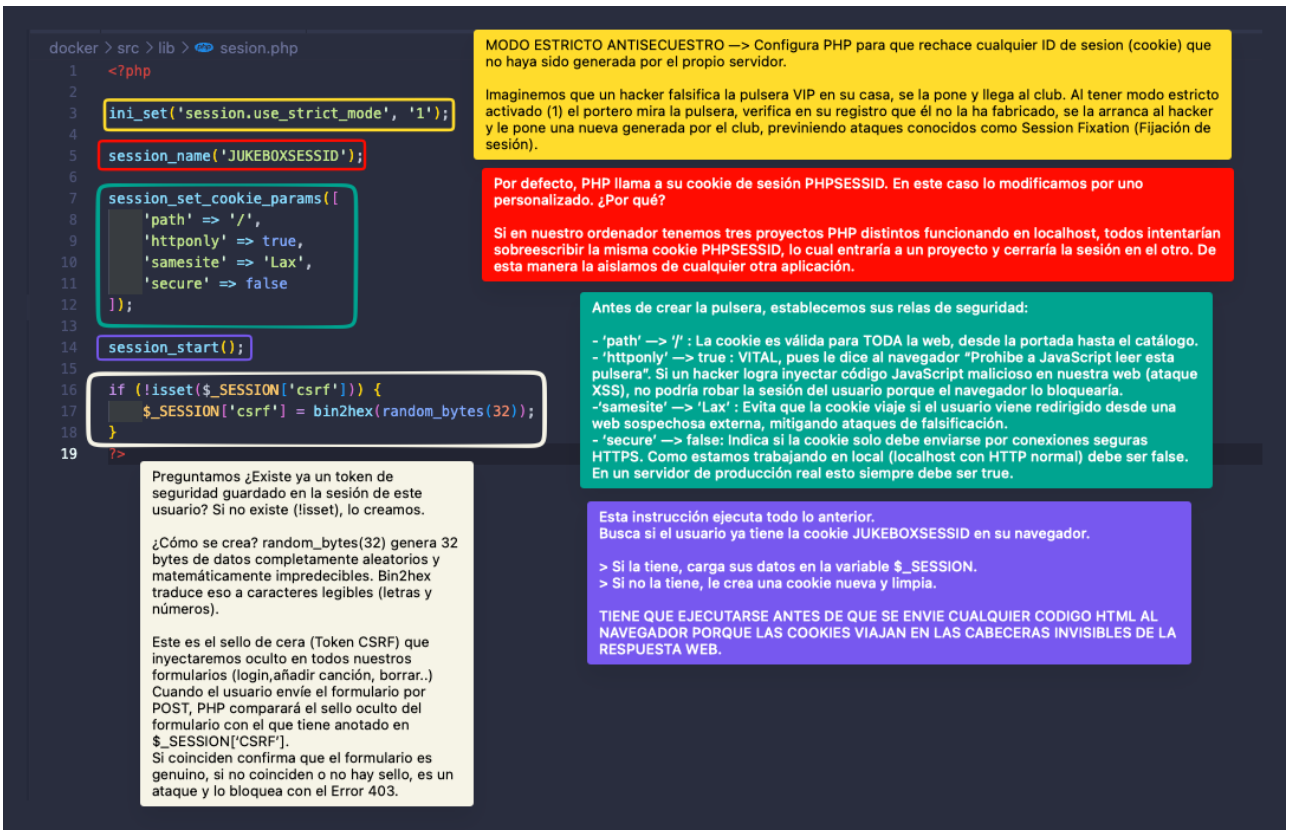
- El Token CSRF: Imaginemos que un hacker crea una web falsa con un formulario invisible que apunta a nuestro servidor para intentar borrar canciones en nuestro nombre. Para evitar esto, a cada usuario le damos una contraseña secreta temporal (sello de cera). Cada vez que el usuario nos envíe un formulario (POST), debe incluir ese sello. Si el sello no coincide con el que le dimos, rechazamos el formulario (ERROR 403).

CREAMOS EL "GESTOR DE LA PUERTA" (lib/sesion.php)

Crearemos el archivo encargado de poner la pulsera (cookie) y generar el sello de cera (token CSRF).

Este archivo deberá cargarse al principio de todas las páginas privadas que hagamos a partir de ahora.

En docker/src/lib creamos el archivo sesion.php



CREAMOS EL FORMULARIO BLINDADO

Crearemos un archivo temporal para probar el funcionamiento de lo que hemos desarrollado. Es un "simulador de ataques" cuyo único objetivo es demostrar empíricamente que el mecanismo del token CSRF (sello de cera) funciona antes de aplicarlo a los formularios reales de registro y login de nuestra aplicación real.




```php
<?php
require_once __DIR__ . '/lib/sesion.php';

$mensaje = "";

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $tokenRecibido = $_POST['csrf'] ?? '';

    if (!hash_equals($_SESSION['csrf'], $tokenRecibido)) {
        http_response_code(403);
        echo "<h1 style='color:red;'>Error 403: Intento de falsificación detectado (CSRF inválido)</h1>";
        exit; /
    }

    $mensaje = "¡Formulario aceptado! El sello CSRF es válido.";
}
?>

<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Prueba de Seguridad CSRF</title>
</head>
<body>
    <h1>Prueba del Escudo CSRF</h1>

    <p>Tu sello secreto actual es: <strong><?= $_SESSION['csrf'] ?></strong></p>

    <?php if ($mensaje): ?>
        <p style="color:green; font-weight:bold;"><?= $mensaje ?></p>
    <?php endif; ?>

    <form method="post">
        <label>Escribe algo: <input type="text" name="dato"></label>

        <!-- ESTO ES VITAL: Inyectamos el sello en un input invisible -->
        <input type="hidden" name="csrf" value="<?= htmlspecialchars($_SESSION['csrf'], ENT_QUOTES, 'UTF-8') ?>">

        <button type="submit">Enviar Seguro</button>
    </form>
</body>
</html>
```

Probamos su ejecución:

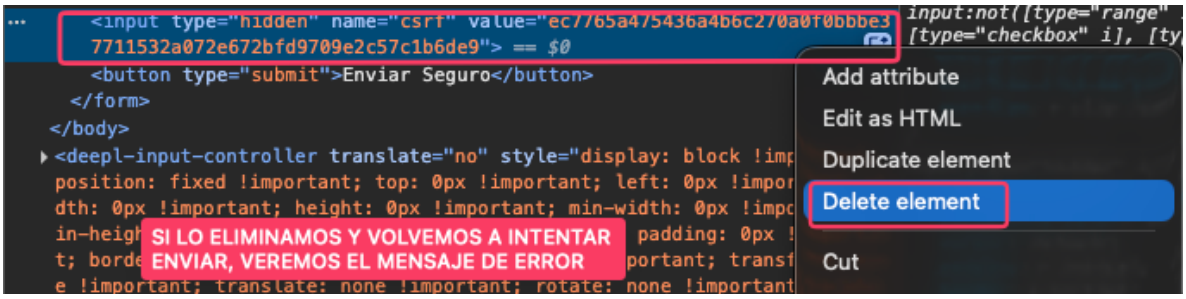


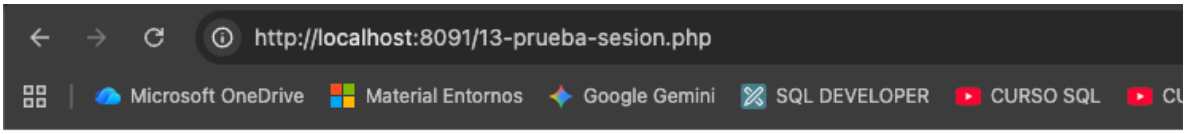
Prueba del Escudo CSRF

Tu sello secreto actual es: **ec7765a475436a4b6c270a0f0bbbe37711532a072e672bfd9709e2c57c1b6de9**

¡Formulario aceptado! El sello CSRF es válido.

Escribe algo: Enviar Seguro





Error 403: Intento de falsificación detectado (CSRF inválido)

REGISTRO DE USUARIOS

En esta sección aprenderemos como los sistemas profesionales protegen las contraseñas reales y evitan que dos personas intenten escribir en el mismo archivo a la vez.

Para entender esta sección, usaremos la siguiente analogía:

La “trituradora de carne” para las contraseñas

- Hash de contraseñas (password_hash) → NUNCA, se guarda la contraseña de un usuario tal cual en la base de datos como texto plano.

  Lo que hacemos es pasar su contraseña por un algoritmo matemático (como una trituradora de carne).
  Ej: Entra “123456” y sale un texto incomprensible (ej: $2y$10$abce…)

Lo “mágico” de esta trituradora es que funciona EN UN SOLO SENTIDO: No se puede tomar la carne picada (hash) y reconstruir la contraseña original, por lo que de nada le sirve a un hacker robar el JSON porque verá hashes inútiles.

El “pestillo” para los turnos

- Bloqueo de archivos (Flock) → Imaginemos que dos administradores intentan registrar una canción o un usuario en el mismo exacto milisegundo.

  Si ambos abren el archivo JSON a la vez, se sobrescribirán y perderemos datos. Para evitarlo usamos Flock() que quiere decir “File lock”.

  Funciona exactamente igual que el pestillo de un lavabo: El primero que llega, pone el pestillo, y el segundo tiene que esperar en la puerta hasta que el primero termine de escribir, guarde y quite el pestillo.

➢ CREAMOS LA BASE DE DATOS DE USUARIOS

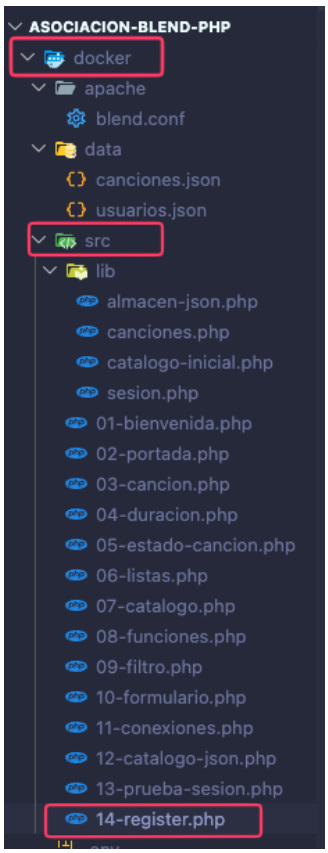
A) Al igual que antes creamos un listado de canciones (canciones.json), en el mismo directorio crearemos un archivo llamado usuarios.json, con una lista vacía.



B) Crearemos una pantalla de registro (register.php) para que la carga de los usuarios se haga mediante una interfaz en el navegador.

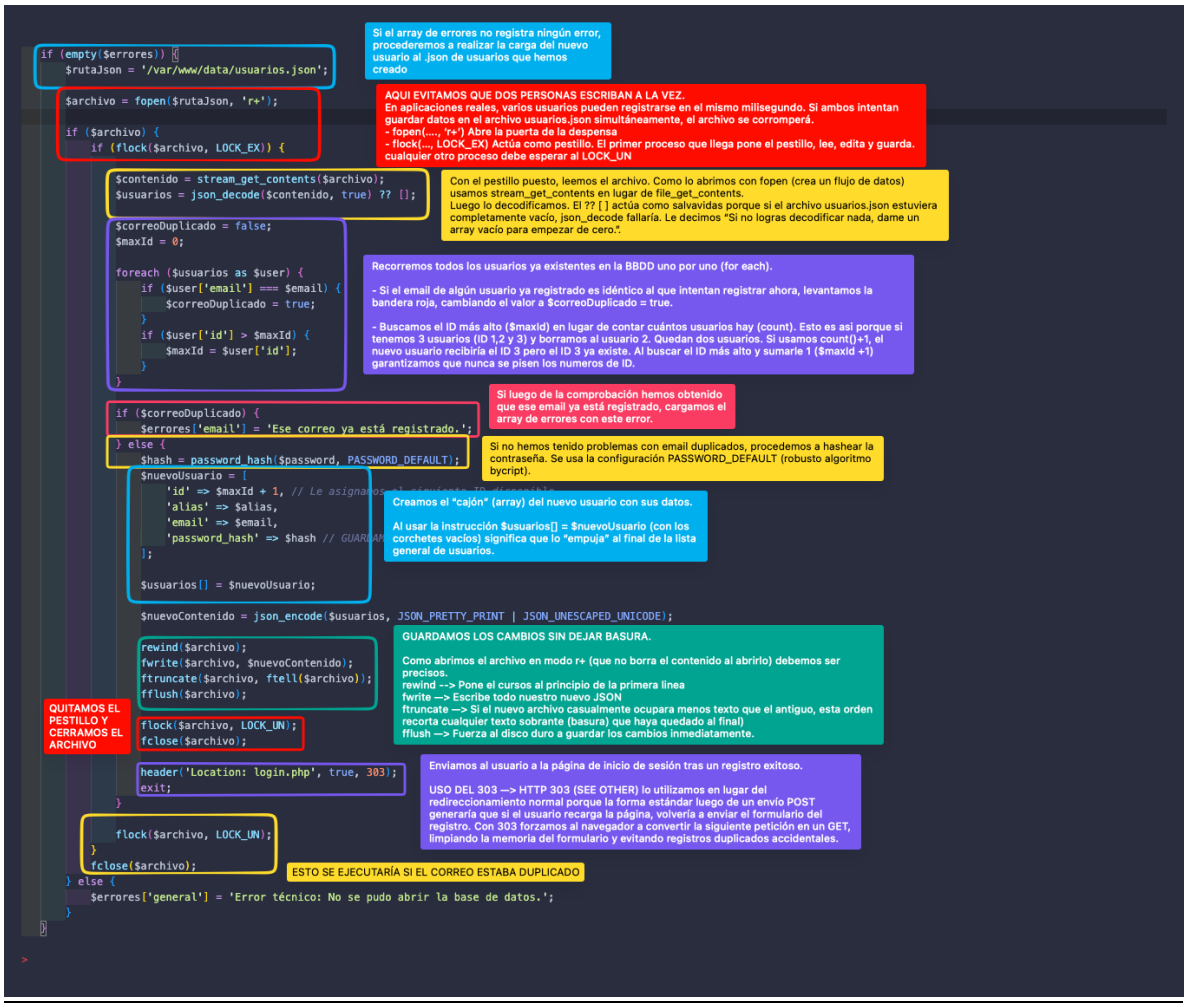
Este archivo, tendrá todo lo que venimos analizando hasta ahora:

- Protección CSRF
- Validación de datos
- Hasheo de contraseñas
- Escritura segura con el “pestillo” (Flock)



Análisis del código:

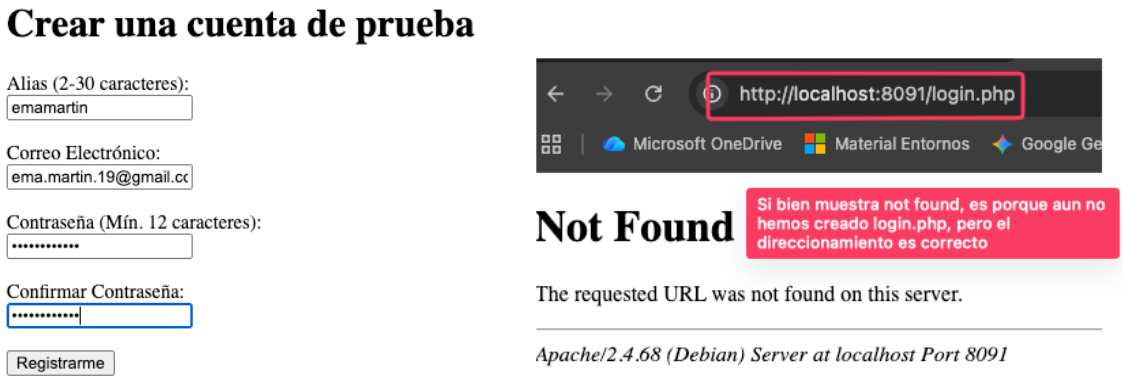

docker > src > 14-register.php
Llamamos a la función que creamos anteriormente ya que la usaremos para las validaciónes de sesión.
Apenas se dispara el metodo POST (El usuario hace click en registrar) se empieza con los controles
PRIMER CONTROL DE SEGURIDAD:
Antes de siquiera mirar que datos ha enviado el usuario, verificamos la "marca de agua" de seguridad.
Si un atacante intenta enviar datos desdse una web externa hacia nuestro servidor, NO TENDRA EL TOKEN GENERADO POR SESION.PHP. Por lo que al realizar la comparación del token recibido con el guardad utilizando la función criptográfica hash_equals, bloquearemos cualquier ataque de suplantación de identidad, disparando el error 403.
RECOGIDA Y LIMPIEZA DE LOS DATOS INTRODUCIDOS
- trim(...) elimina los espacios en blanco accidentales que el usuario haya podido teclear al pincipio o al final.
-strtolower() para el correo eléctronico porque para un ordenador "Juan@Test.com" y "juan@test.com" son distintos. Al forzar todo a minúsculas evitamos que un usuario se registre dos veces por error con las mayúsculas cambiadas.
ZONA DE VALIDACION DE DATOS INTRODUCIDOS
AQUÍ COMPROBAMOS QUE LOS DATOS CUMPLEN CON LAS REGLAS DE NEGOCIO
- mb_strlen (multi-byte String Length) en lugar del simple strlen para el alias, nos asegura de que cuente correctamente caracteres especiales como los acentos o las eñes.
- Para el correo electrónico utlizamos filter_var con (FILTER_VALIDATE_EMAIL) que completa matemáticamente si el formato introducido corresponde a un email real (con su arroba y su dominio).
- En el password si utlizamos el strlen tradicional, para validar que la contraseña tiene más de 12 y menos de 64 caracteres
Cada vez que "falla" una comprobación" se carga el array vacío de errores que habíamos creado con una propiedad (el campo que ha fallado) y un valor (un mensaje de error).


Si el array de errores no registra ningún error, procederemos a realizar la carga del nuevo usuario al .json de usuarios que hemos creado
AQUI EVITAMOS QUE DOS PERSONAS ESCRIBAN A LA VEZ.
En aplicaciones reales, varios usuarios pueden registrarse en el mismo milisegundo. Si ambos intentan guardar datos en el archivo usuarios.json simultáneamente, el archivo se corromperá.
- fopen(..., 'r+') Abre la puerta de la despensa
- flock(..., LOCK_EX) Actúa como pestillo. El primer proceso que llega pone el pestillo, lee, edita y guarda. cualquier otro proceso debe esperar al LOCK_UN
Con el pestillo puesto, leemos el archivo. Como lo abrimos con fopen (crea un flujo de datos) usamos stream_get_contents en lugar de file_get_contents.
Luego lo decodificamos. El ?? [ ] actúa como salvavidas porque si el archivo usuarios.json estuviera completamente vacío, json_decode fallaría. Le decimos "Si no logras decodificar nada, dame un array vacío para empezar de cero.".
Recorremos todos los usuarios ya existentes en la BBDD uno por uno (for each).
- Si el email de algún usuario ya registrado es idéntico al que intentan registrar ahora, levantamos la bandera roja, cambiando el valor a $correoDuplicado = true.
- Buscamos el ID más alto ($maxId) en lugar de contar cuántos usuarios hay (count). Esto es asi porque si tenemos 3 usuarios (ID 1,2 y 3) y borramos al usuario 2. Quedan dos usuarios. Si usamos count()+1, el nuevo usuario recibiría el ID 3 pero el ID 3 ya existe. Al buscar el ID más alto y sumarle 1 ($maxId +1) garantizamos que nunca se pisen los numeros de ID.
Si luego de la comprobación hemos obtenido que ese email ya está registrado, cargamos el array de errores con este error.
Si no hemos tenido problemas con email duplicados, procedemos a hashear la contraseña. Se usa la configuración PASSWORD_DEFAULT (robusto algoritmo bycript).
Creamos el "cajón" (array) del nuevo usuario con sus datos.
Al usar la instrucción $usuarios[] = $nuevoUsuario (con los corchetes vacíos) significa que lo "empuja" al final de la lista general de usuarios.
GUARDAMOS LOS CAMBIOS SIN DEJAR BASURA.
Como abrimos el archivo en modo r+ (que no borra el contenido al abrirlo) debemos ser precisos.
rewind --> Pone el cursos al principio de la primera linea
fwrite —> Escribe todo nuestro nuevo JSON
ftruncate —> Si el nuevo archivo casualmente ocupara menos texto que el antiguo, esta orden recorta cualquier texto sobrante (basura) que haya quedado al final)
fflush —> Fuerza al disco duro a guardar los cambios inmediatamente.
QUITAMOS EL PESTILLO Y CERRAMOS EL ARCHIVO
Enviamos al usuario a la página de inicio de sesión tras un registro exitoso.
USO DEL 303 —> HTTP 303 (SEE OTHER) lo utilizamos en lugar del redireccionamiento normal porque la forma estándar luego de un envío POST generaría que si el usuario recarga la página, volvería a enviar el formulario del registro. Con 303 forzamos al navegador a convertir la siguiente petición en un GET, limpiando la memoria del formulario y evitando registros duplicados accidentales.
ESTO SE EJECUTARÍA SI EL CORREO ESTABA DUPLICADO

PRUEBAS DE FUNCIONAMIENTO

- PRUEBAS VÁLIDAS

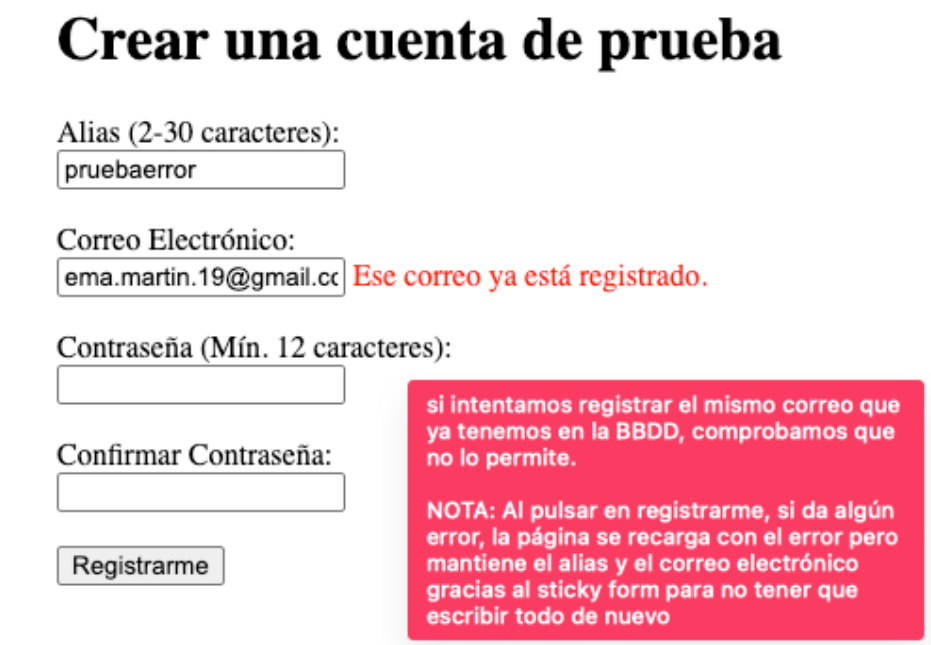
Para la comprobación válida, nos registraremos con datos correctos.





- PRUEBAS DE RECHAZO

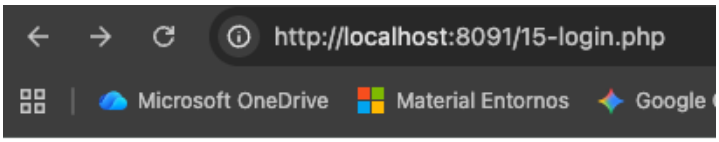




PERMITIR ENTRAR A LOGIN.PHP A LOS USUARIOS YA REGISTRADOS

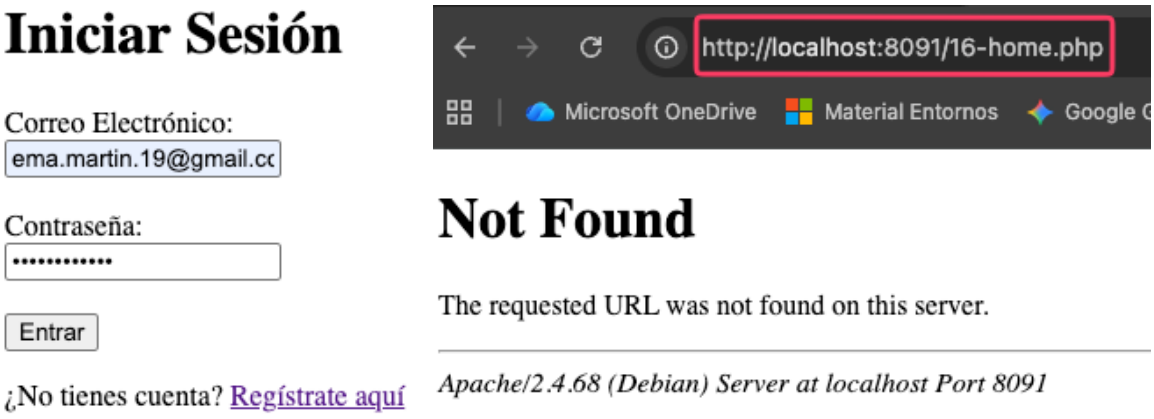
Cuando la registración es exitosa, el usuario es derivado a la página login.php.

Crearemos ese archivo, el cual tendrá una estructura muy similar a la anterior. Añadiremos al verificador de credenciales la función password_verify que permite comprobar si la contraseña coincide con el hash sin necesidad de descifrarlo.





Un acceso exitoso en login, nos debe redirigir automáticamente al archivo 16-home.php. Como aún no hemos creado este archivo, nos mostrará un mensaje de error pero la actualización de la URL será correcta.



Datos erróneos:

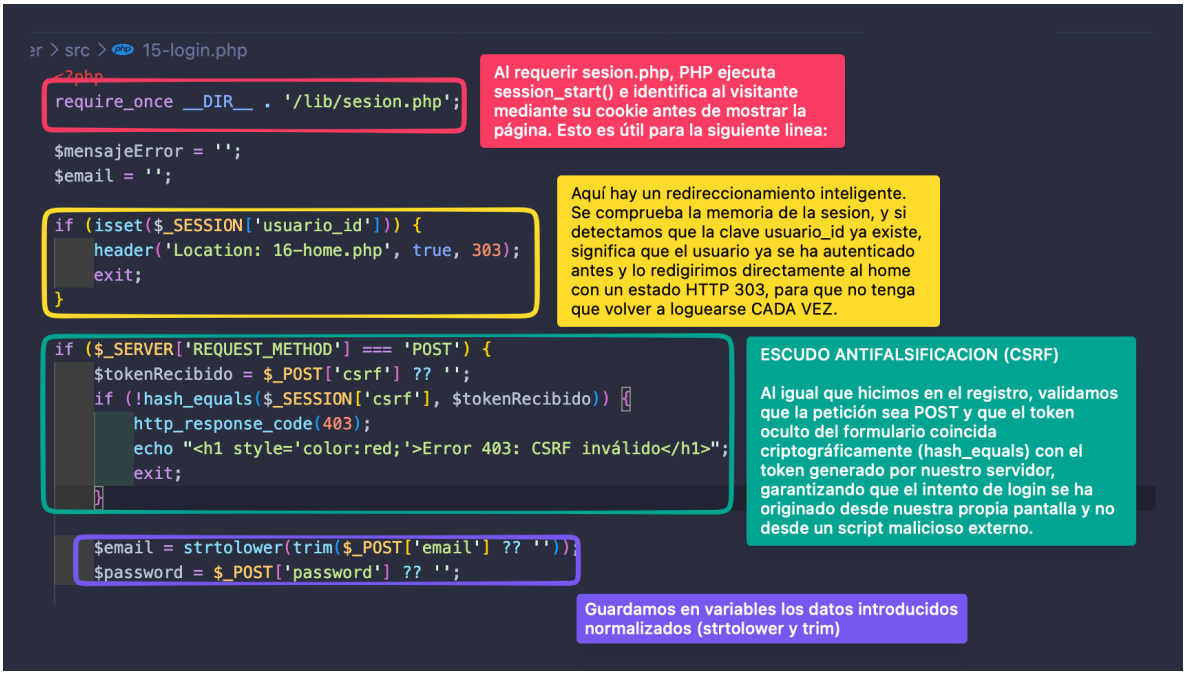


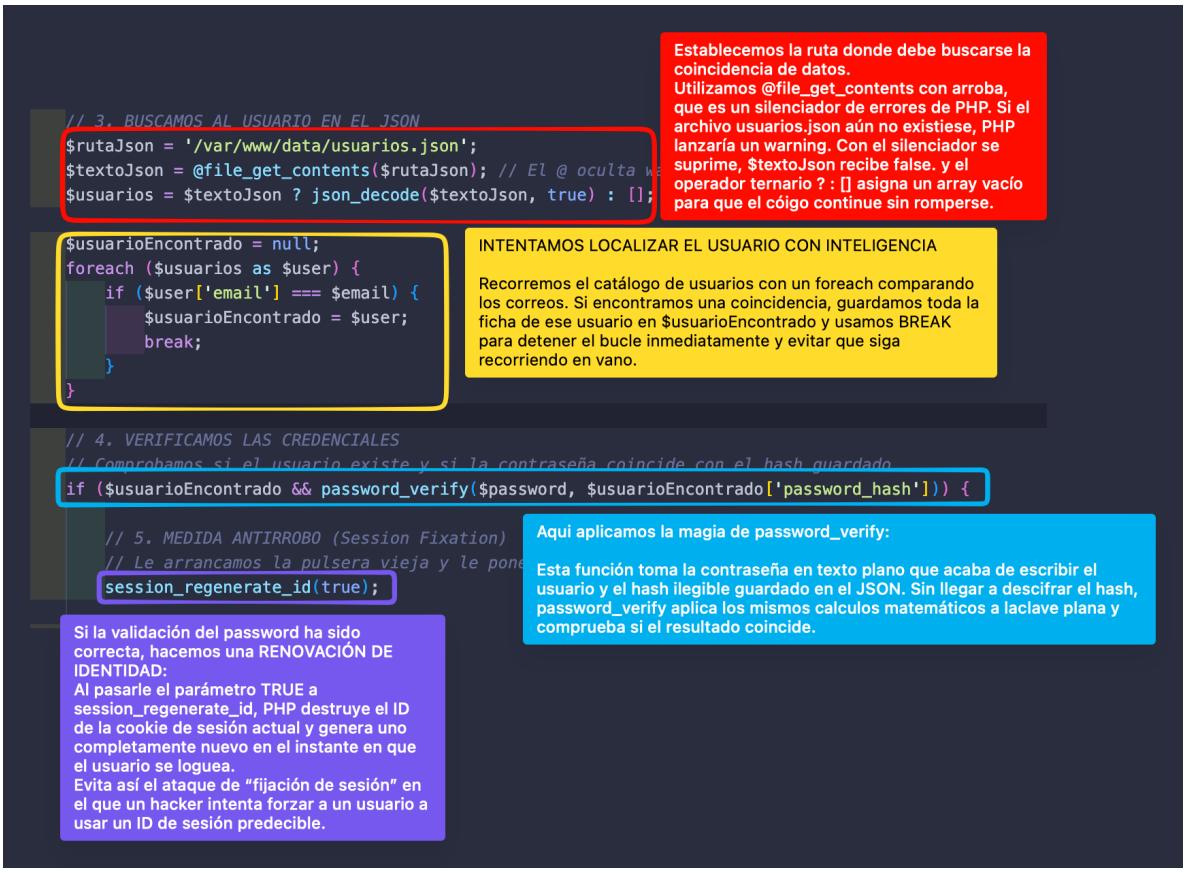
Además hemos añadido un enlace para ir directamente al formulario de registro:

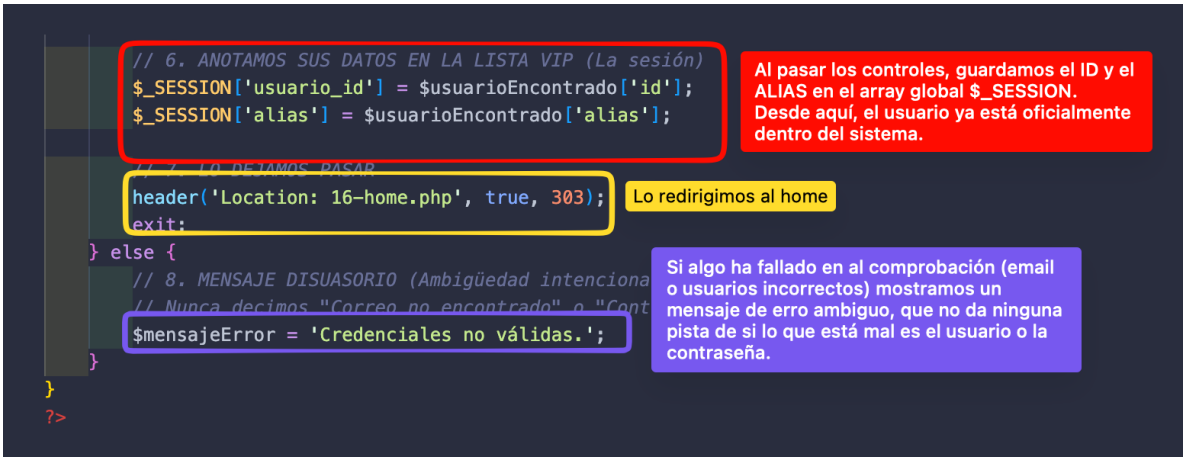


Explicación a nivel de código:

Este archivo login.php actúa como control de aduanas de la aplicación. Su objetivo no es solo verificar si la contraseña es correcta, sino hacerlo de una forma que el atacante no obtenga ninguna pista útil si intenta forzar la entrada.



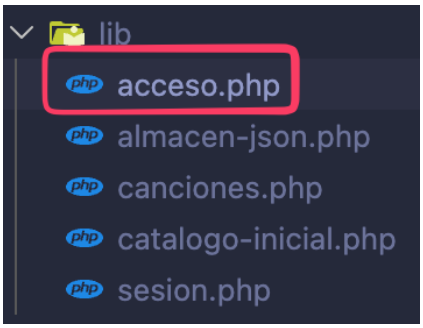




PROTECCIÓN DE "ZONAS VIP" – ARCHIVO ACCESO.PHP

Crearemos un archivo que sirva como "seguridad" en las zonas privadas. Será llamado al principio de cualquier página que requiera estar logueado (como por ejemplo el home o el editor de canciones).

Como ocultar enlaces no protege a una URL, si alguien escribe directamente localhost:8091/16-home.php en el navegador, PHP debe detenerlo. Este archivo se encargará específicamente de eso y lo incluiremos al inicio de cada página privada.





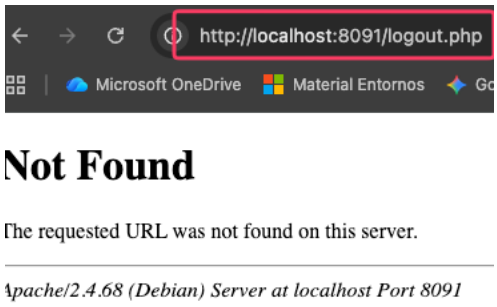
Si no ha ejecutado el if, el script simplemente termina y deja continuar.

CREACIÓN DE LA HOME Y BOTÓN DE CERRAR SESIÓN

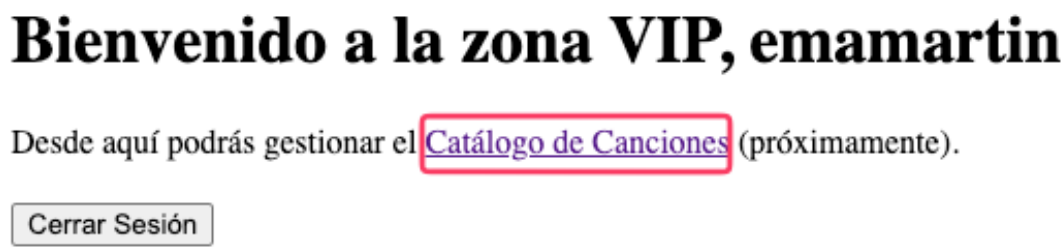
Ya hemos especificado al login.php que un vez que se han comprobado las credenciales correctamente debe redirigir al usuario a la página 16-home.php

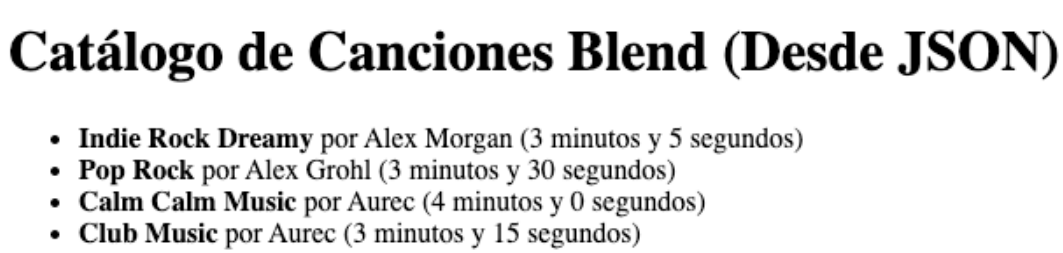
Así mismo, hasta ahora una vez que entrabamos, se guardaban los datos de la sesión en la cookie y cada vez que intentábamos entrar en login.php directamente nos enviaba a la home. Solo podíamos borrar los datos de navegación (borrando la cookie) para poder volver a acceder a ella. Ahora crearemos un botón de cerrar sesión que nos liberará de eso, llevandonos a logout.php.





Asimismo, en el home tenemos un enlace que nos lleva directamente al catálogo de canciones:





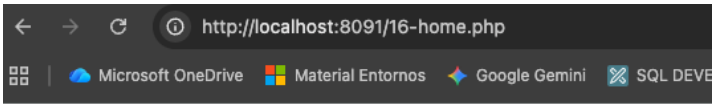


CREACIÓN DEL LOGOUT

Hasta el momento teníamos el botón de cerrar sesión sin funcionar. Tenemos que darle la funcionalidad que corresponde: Esconder trampas de seguridad muy interesantes.

Cerrar sesión no es solo echar al usuario, es destruir por completo cualquier rastro de su identidad para que nadie pueda reutilizar su pulsera.



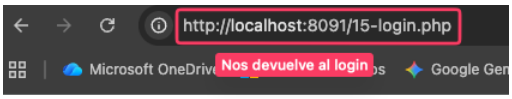


Bienvenido a la zona VIP, emamartin

Desde aquí podrás gestionar el Catálogo de Canciones (próximamente).

Cerrar Sesión

Ahora al cerrar sesión...



Iniciar Sesión

Correo Electrónico:

Contraseña:

Entrar

¿No tienes cuenta? Regístrate aquí

11) CREACIÓN DEL FORMULARIO

En los bloques anteriores, creamos una previsualización de un formulario (10-formulario.php) que nos permitía hacer una previsualización de lo que sería la subida de una canción que no iba realmente hacia ninguna parte.



Ahora crearemos el formulario oficial. Cuando el usuario haga click en “Guardar”, PHP tomará los datos, irá hacia la despensa segura que hemos montado (docker/data/canciones.json), colocará el “pestillo” (Flock) para que ningún otro administrador pueda añadir una canción en el mismo milisegundo, abrirá la caja de canciones, ingresará la nueva ficha, cerrará la caja y quitará ese pestillo, para luego llevarnos nuevamente al catálogo y visualizar el resultado.

11.1) CREANDO EL “CREADOR DE CANCIONES”

En docker/src, creamos un nuevo archivo llamado 18-nueva-cancion.php. En este archivo incluiremos tres cosas:

- Formulario visual
- Validaciones de errores
- Motor de guardado seguro

src > 18-nueva-cancion.php

```php
<?php
// SEGURIDAD DE ENTRADA. Solo usuarios logueados p
require_once __DIR__ . '/lib/acceso.php';

// FORMATEADOR DE SEGUNDOS
require_once __DIR__ . '/lib/canciones.php';

$errores = [];
$title = '';
$artist = '';
$category = '';
$duration = '';
$src = '';

// ¿EL USUARIO HA PULSADO EL BOTÓN DE GUARDAR?
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // A. EL ESCUDO CSRF (Marca de agua)
    $tokenRecibido = $_POST['csrf'] ?? '';
    if (!hash_equals($_SESSION['csrf'], $tokenRecibido)) {
        http_response_code(403);
        echo "<h1 style='color:red;'>Error 403: Petición rechazada (CSRF inválido)</h1>";
        exit;
    }
```

INCORPORAMOS DOS HERRAMIENTAS:

1) ACCESO .PHP QUE NOS ASEGURARÁ QUE SOLO LOS USUARIOS LOGUEADOS PUEDAN VER ESTA PÁGINA, EVITANDO QUE ALGUIEN SOLO CON LA URL PUEDA ENTRAR. LOS REDIRIGE AL LOGIN SI NO ESTAN LOGUEADOS.

2) CANCIONES.PHP NOS TRAERA LA FUNCIÓN FORMATEADORA DE SEGUNDOS A MINUTOS Y SEGUNDOS

Inicializamos el array de errores para que se vaya cargando a media que se detecten y las variables que en las que iremos guardando los datos ingresados luego de pasar la validación de errores.

Verificamos si el usuario ha pulsado guardar, estableciendo el metodo seguro POST para inicializar el proceso de validación y guardado de la cancion.

Verificamos si la clave CSRF enviada al servidor (tokenRecibido) coincide con la que tiene guardada el servidor en la sesión.

Si no hay coincidencias, imprime mensaje de error y el exit no permite seguir, directamente bloquea la página con el mensaje de error.

```php
    // B. RECOGIDA Y LIMPIEZA DE LOS DATOS
    $title = trim($_POST['title'] ?? '');
    $artist = trim($_POST['artist'] ?? '');
    $category = $_POST['category'] ?? '';
    $duration = $_POST['duration'] ?? '';
    $src = $_POST['src'] ?? '';

    // C. VALIDACIÓN (Control de calidad)
    if ($title === '') {
        $errores['title'] = 'El título es obligatorio.';
    }
    if ($artist === '') {
        $errores['artist'] = 'El artista es obligatorio.';
    }

    $allowedCategories = ['electronica', 'rock', 'relax'];
    if (!in_array($category, $allowedCategories, true)) {
        $errores['category'] = 'Categoría no válida.';
    }

    $duracionInt = filter_var($duration, FILTER_VALIDATE_INT);
    if ($duracionInt === false || $duracionInt < 1 || $duracionInt > 1800) {
        $errores['duration'] = 'La duración debe ser un número entre 1 y 1800 segundos.';
    }

    $allowedSrc = ['audio/demo-01.mp3', 'audio/demo-02.mp3'];
    if (!in_array($src, $allowedSrc, true)) {
        $errores['src'] = 'El archivo de audio no es válido.';
    }
```

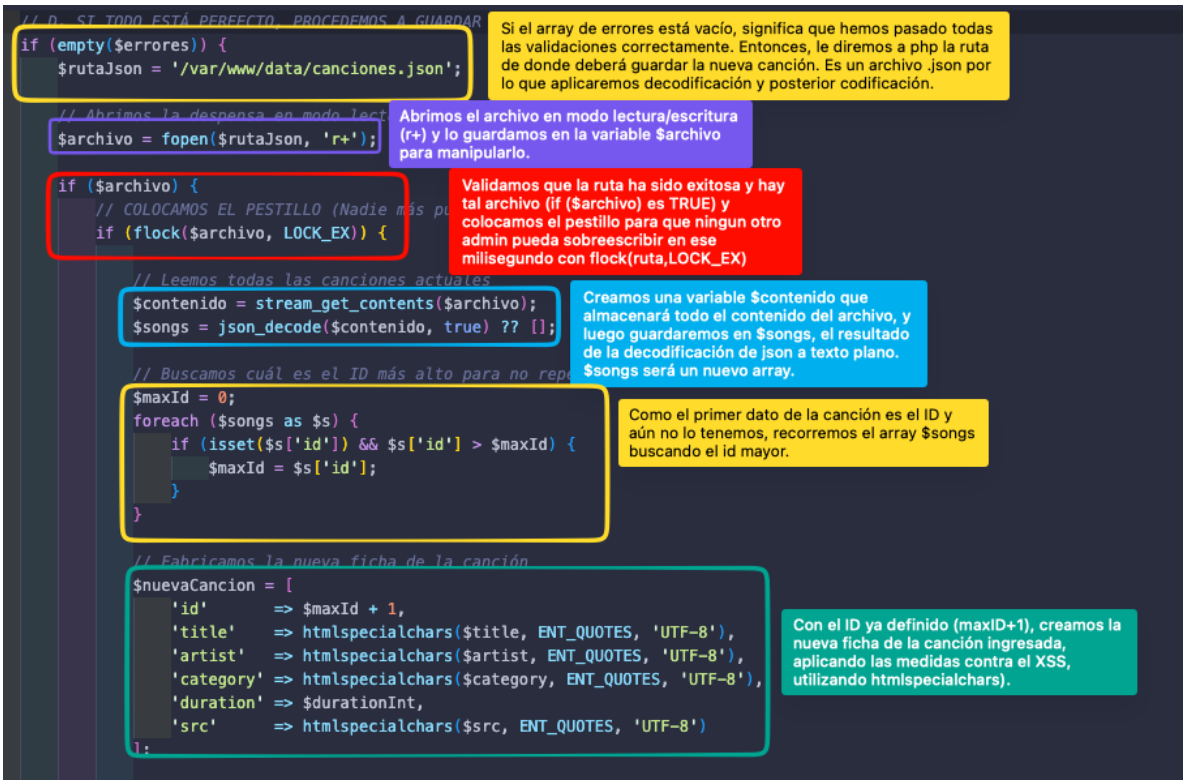
Recogemos los datos enviados en la petición POST y los guardamos en las variables correspondientes que hemos creado, utilziando trim en title y artist para eliminar espacios en blanco y añadiendo el ?? '' para evitar que si un dato viene vacío o dañado se corrompa la página, asignando un valor null.

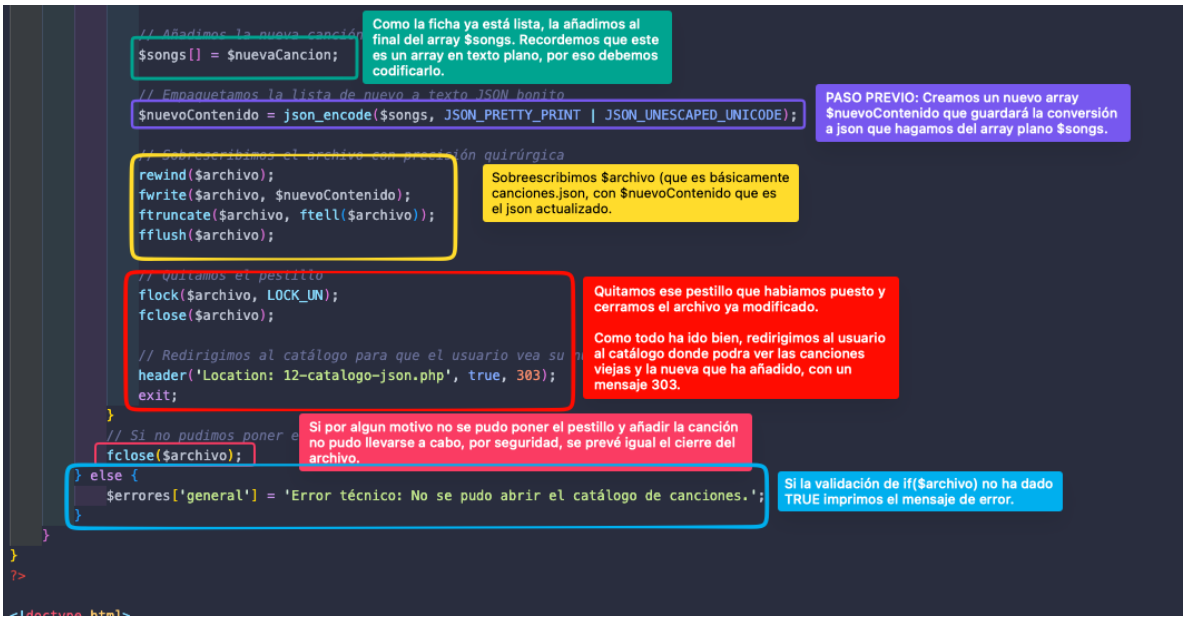
Como el titulo y el artista no pueden estar vacíos (obligatorios), determinamos que si tienen valor NULL se cargue el array de errores con el mensaje correspondiente

Analizamos si la categoría seleccionada está dentro de las listadas en el array de disponibles. En caso contrario, mostrará un mensaje de error.

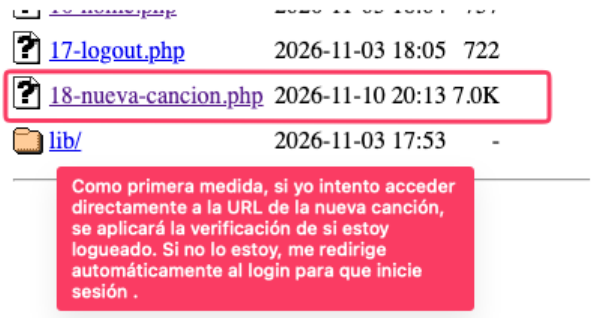
Para la duración usamos un filtro para identificar si el valor ingresado es un entero, para que en caso de que no lo sea (primer condicion del if), de un error. Además, si es entero pero menor que 1 y mayor que 1800, tampoco pasará la validación.

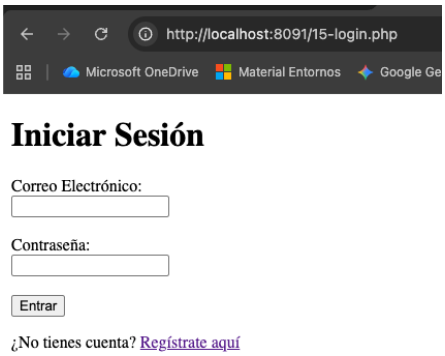
Por ahora solo utilizaremos recursos de ejemplo para los archivos, por lo que solo se validará si la opción elegida coincide con alguna del array de opciones desplegado.





Pruebas visuales:

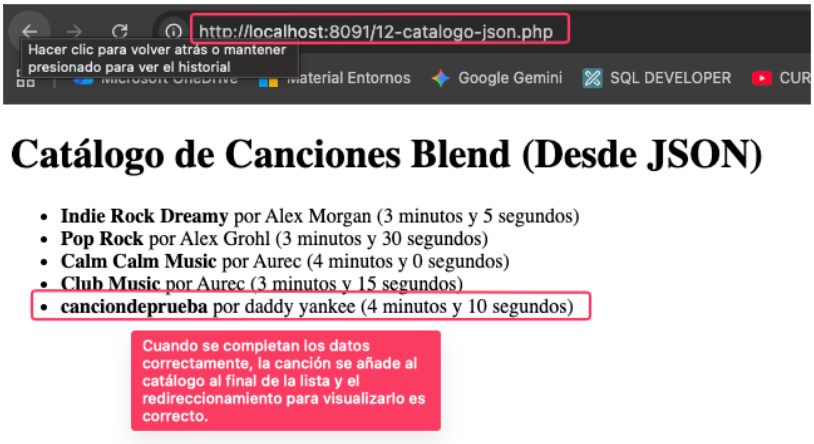

17-logout.php 2026-11-03 18:05 722
18-nueva-cancion.php 2026-11-10 20:13 7.0K
lib/ 2026-11-03 17:53 -
Como primera medida, si yo intento acceder directamente a la URL de la nueva canción, se aplicará la verificación de si estoy logueado. Si no lo estoy, me redirige automáticamente al login para que inicie sesión .


http://localhost:8091/15-login.php
Microsoft OneDrive
Material Entornos
Iniciar Sesión
Correo Electrónico:
Contraseña:
Entrar
¿No tienes cuenta? Regístrate aquí


http://localhost:8091/18-nueva-cancion.php
Microsoft OneDrive
Material Entornos
Google Gemini
Añadir Nueva Canción al Catálogo
← Volver a la Home VIP
Título:
Artista:
Una vez que he iniciado sesión, puedo acceder correctamente al nuevo formulario de creación.
Categoría:
Selecciona...
Duración (segundos):
Archivo de audio:
Selecciona...
Guardar Canción Oficialmente

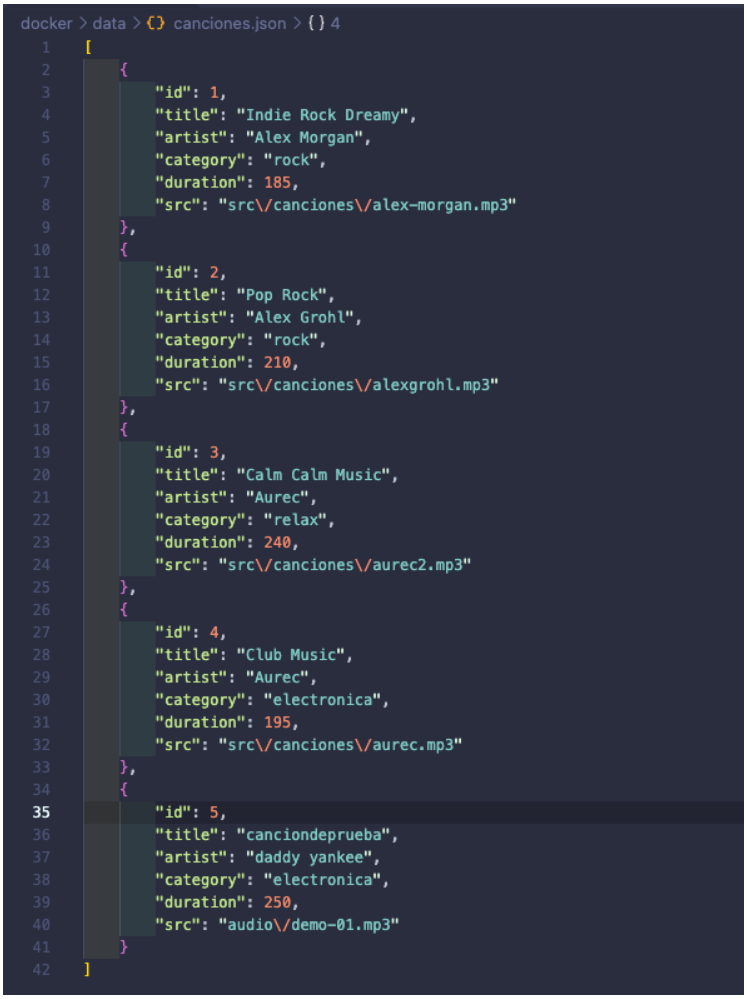
Ç


http://localhost:8091/18-nueva-cancion.php
Microsoft OneDrive
Material Entornos
Google Gemini
SQL DEVELOPER
Añadir Nueva Canción al Catálogo
← Volver a la Home VIP
Al intentar enviar sin llenar los campos, verificamos que las reglas de validación estan funcionando.
Título:
El título es obligatorio.
Artista:
El artista es obligatorio.
Categoría:
Selecciona...
Categoría no válida.
Duración (segundos):
0
La duración debe ser un número entre 1 y 1800 segundos.
Archivo de audio:
Selecciona...
El archivo de audio no es válido.
Guardar Canción Oficialmente



12) DEL ARCHIVO JSON A LA BASE DE DATOS

Hasta ahora y tal como vimos en el bloque anterior, nuestra base de datos de canciones es un archivo.json llamado canciones.json.



Sin embargo, un archivo.json funciona como una libreta de papel. Si tenemos 10000 canciones y queremos buscar una de rock, PHP tiene que abrir la libreta, leer página por página (con un bucle foreach) hasta encontrarla. Esto es lento y poco profesional.

Para solucionar este problema, incorporaremos las bases de datos relacionales (PostgreSQL/MariaDB) que encendimos anteriormente con el docker compose.

¿Qué soluciona esto? → Es un archivo digital inteligente. En lugar de leer página por página, PHP le dice a la BBDD "Busca todas las canciones de rock que duren menos de 3 minutos" (consulta SQL) y el archivero le devuelve el resultado en un milisegundo.

Para utilizar estos archiveros, necesitamos crear CAJONES ("TABLAS") en las cuales guardaremos las fichas.

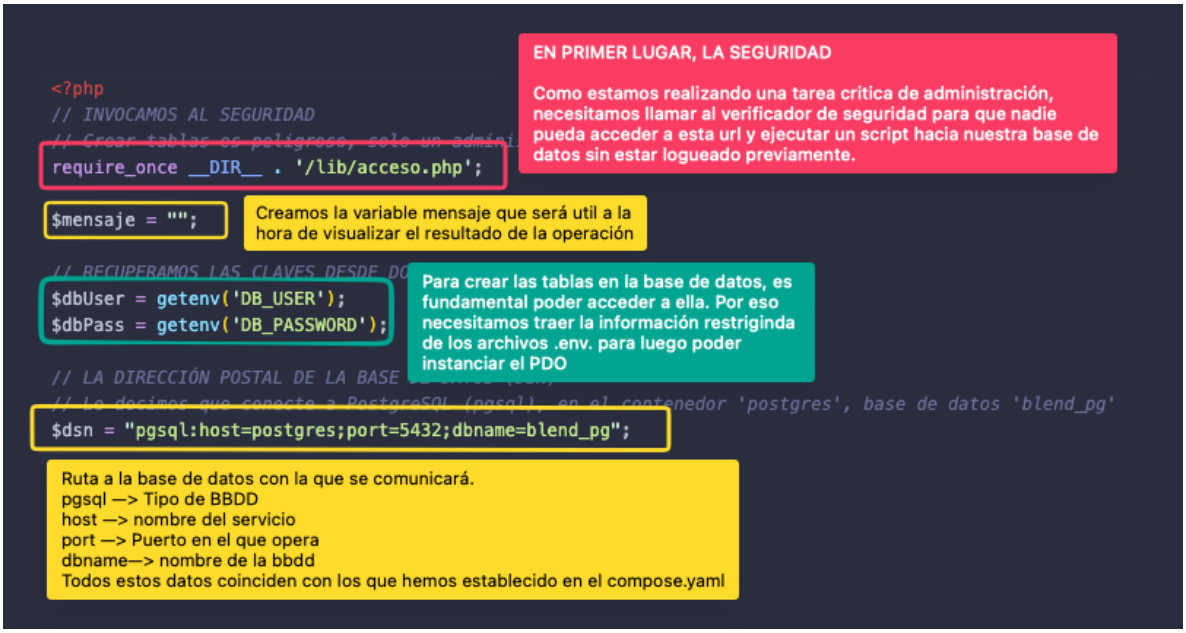
PRIMER PASO: CONSTRUIR LAS TABLAS

En este caso no entraremos directamente a la base de datos a crear las tablas a mano. Sino que escribiremos un script en PHP para que lo haga automáticamente por nosotros utilizando:

- PDO (PHP DATA OBJECTS)

PDO es un "traductor universal" que permite hablar con "PostgreSQL" o "MariaDB" utilizando exatamente el mismo código en PHP.

Crearemos un nuevo archivo llamado 19-setup-db.php.



```yaml
postgres:
  image: postgres:17-bookworm
  environment:
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: blend_pg
  volumes:
    - pg_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d blend_pg"]
    interval: 5s
    timeout: 5s
    retries: 5
```

```
NAME                                    IMAGE                          COMMAND                  SERVICE    CREATED      STATUS                    PORTS
asociacion-blend-multiservicio-mariadb-1    mariadb:11.4                   "docker-entrypoint.s…"   mariadb    7 days ago   Up 7 days (healthy)       3306/tcp
asociacion-blend-multiservicio-postgres-1   postgres:17-bookworm           "docker-entrypoint.s…"   postgres   7 days ago   Up 7 days (healthy)       5432/tcp
asociacion-blend-multiservicio-web-1        asociacion-blend-multiservicio-web   "docker-php-entrypoi…"   web        7 days ago   Up 7 days                 127.0.0.1:8091->80/tcp
emamartin@MacBook-Air-de-Emanuel docker %
```

FUNDAMENTAL ESTABLECER UN TRY CATCH PARA CAPTURAR POSIBLES ERRORES

```php
try {
    // ABRIMOS LA CONEXIÓN (El teléfono directo con el Archivero)
    $pdo = new PDO($dsn, $dbUser, $dbPass);

    // Configuramos el teléfono para que nos grite si hay cualquier error SQL
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // ESCRIBIMOS LAS INSTRUCCIONES SQL (El plano del cajón)
    // Usamos CREATE TABLE IF NOT EXISTS para que no dé error si ya estaba creada
    $sqlCanciones = "
        CREATE TABLE IF NOT EXISTS canciones (
            id SERIAL PRIMARY KEY,
            title VARCHAR(150) NOT NULL,
            artist VARCHAR(150) NOT NULL,
            category VARCHAR(50) NOT NULL,
            duration INT NOT NULL,
            src VARCHAR(255) NOT NULL
        )
    ";

    // ENVIAMOS LA ORDEN AL ARCHIVERO
    // exec() se usa para comandos que no devuelven datos, solo ejecutan acciones
    $pdo->exec($sqlCanciones);

    $mensaje = "¡Éxito! La tabla 'canciones' ha sido creada (o ya existía) en PostgreSQL.";

} catch (PDOException $e) {
    // Si la conexión o la creación falla, capturamos la explosión
    $mensaje = "Error crítico de Base de Datos: " . $e->getMessage();
}
?>
```

Empezamos la conexión, instanciando (new) PDO con los valores que hemos capturado antes. AQUI LLAMAMOS A LA BASE DE DATOS ENTREGANDO LAS CREDENCIALES SECRETAS

MODO DE ERRORES —> Por defecto, si PDO se equivoca, se queda callado. Al activar las excepciones, le ordenamos que si cualquier orden SQL falla (ej nos equivocamos al escribir VARCHAR), se dispara una laarma que es atrapada por el bloque try/catch evitando que la página colapse en blanco y revelando el problema real.

EL SCRIPT MÁGICO PARA CREAR LA TABLA

— CREATE TABLE IF NOT EXISTE canciones —> Es una medida de seguridad. Si recargamos la página 5 veces, el código SQL no intentará crear la tabla 5 veces.
— SERIAL PRIMARY KEY —> En PostgreSQL, SERIAL, es un tipo de dato mágico. Actúa como un contador automático. Cuando añadamos una canción, no tendrá que calcular maxID + 1 como hicimos antes en el JSON, PostgreSQL le asignará el siguiente número disponible automáticamente y garantizará que sea único (PRIMARY KEY).
— VARCHAR(N) E INT —> Definimos la naturaleza de los datos. A diferencia de PHP que es relajado con los tipos, en bases de datos debe ser estricto. Si intentamos guardar texto en la columna duration (INT) la base de datos rechazará la inserción.

EL NOT NULL GARANTIZA QUE NO PUEDEN QUEDAR VACÍOS.

En PDO hay dos formas principales de hablar con una BBDD:
> Preguntas: Ej dime que canciones hay... usamos QUERY() o sentencias preparadas.
> Ordenes que no devuelven ninguna tabla de resultados. Ej. Construye esta tabla... Borra esta fila... usamos EXEC().
Como parámetro se le pasará el script SQL, como en el caso de este codigo.

Damos vida a mensaje, diciendole que si todo ha ido bien y ha llegado a este punto, muestre este mensaje.

Si algo ha ido mal en el proceso, capturamos la excepción, y concatenamos el mensaje de error con el código de error que ha dado la excepcion.

```php
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Instalador de Base de Datos</title>
</head>
<body>
    <h1>Configuración Inicial de la BBDD</h1>

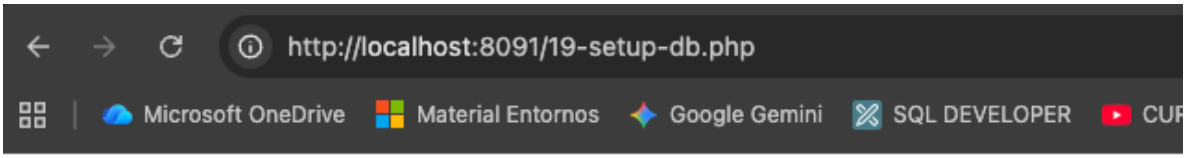
    <!-- Imprimimos el resultado de nuestra operación en verde o rojo -->
    <p style="font-weight: bold; color: <?= strpos($mensaje, 'Error') === false ? 'green' : 'red' ?>;">
        <?= htmlspecialchars($mensaje, ENT_QUOTES, 'UTF-8') ?>
    </p>

    <p><a href="16-home.php">← Volver a la Home</a></p>
</body>
</html>
```

DETALLE: Añadimos un toque de color para mostrar el exito en verde y el error en rojo.

Utilizamos para ello strpos, que escaneara un texto y nos dira en que posición exacta empieza una palabra concreta que buscamos. Si encuentra la palabra, devolverá la posición en la que comienza esa palabra (el número o indice) pero si no la encuentra devolvera FALSE.

Por eso, se establece que si no se encuentra la palabra, el mensaje debe ser verde porque será d eexito, pero si la encuentra, nos da igual la posición, sino que no haya devuelto FALSE para pinterlo en ROJO.



Configuración Inicial de la BBDD

¡Éxito! La tabla 'canciones' ha sido creada (o ya existía) en PostgreSQL.

← Volver a la Home

Al ingresar a la URL ya estando logueados (SUPERAMOS EL CONTROL DE SEGURIDAD) se ha ejecutado el script exitosamente.

SEGUNDO PASO: CREAR EL NUEVO CATALOGO

A partir de ahora abandonaremos el lento archivo .json para hacer que nuestro catálogo lea y guarde la música directamente en la BBDD.

Desarrollaremos el READ (“R”) y el CREATE (“C) del CRUD.

Ventajas que obtendremos y veremos a nivel de código:

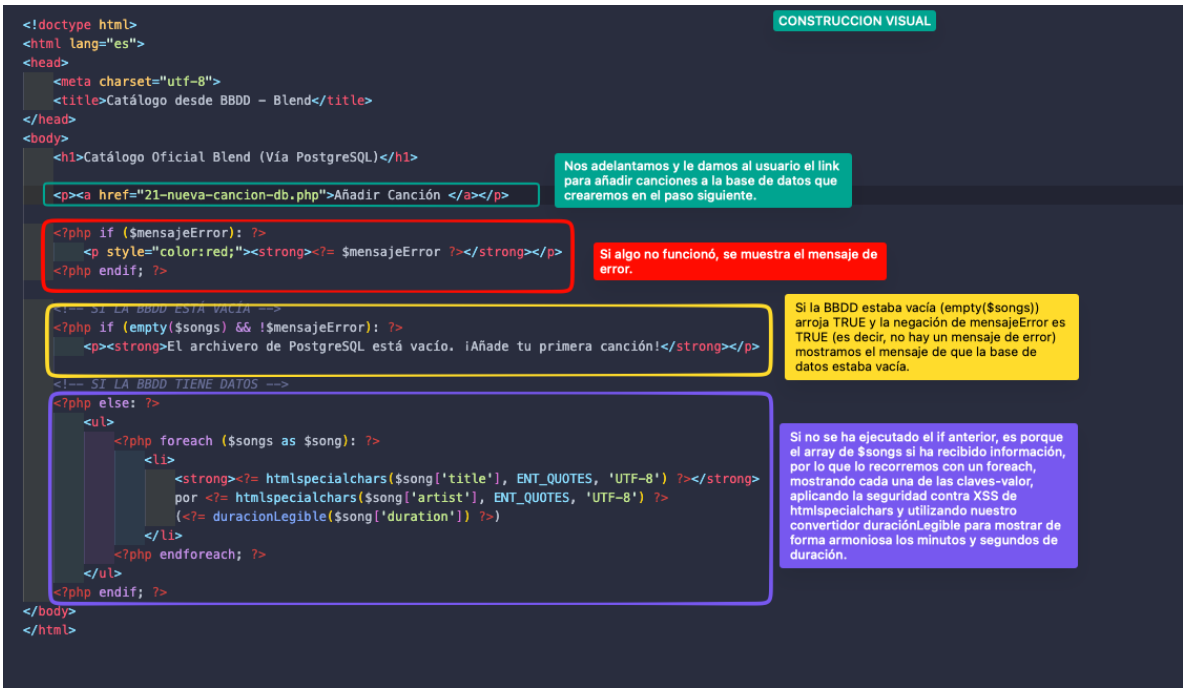
- maxID → Como ya explicamos antes, la base de datos se ocupará de la numeración automática del ID.
- Flock (pestillo) → La base de datos se ocupará por si misma de la concurrencia (varios usuarios a la vez).

Crearemos un nuevo archivo llamado 20-catálogo-db.php que será la página publica que los usuarios van a ver. Sin embargo, en lugar de leer un texto y usar json_decode como

veníamos haciendo, simplemente le pediremos a la base de datos que nos traiga todo el contenido del cajón (tabla).

Notaremos que la estructura es mucho más simple:





Si probamos este código, veremos lo siguiente:



Con la consulta exitosa ya construida, procedemos a desarrollar el nuevo formulario para añadir canciones manualmente, pero estas irán DIRECTAMENTE A LA BASE DE DATOS.

La estructura será muy similar al que ya analizamos anteriormente, lo que variará será el guardado de datos.

src > 21-nueva-cancion-db.php

```php
<?php

// SEGURIDAD: Solo usuarios registrados pueden añadir música
require_once __DIR__ . '/lib/acceso.php';

$errores = [];
$title = '';
$artist = '';
$category = '';
$duration = '';
$src = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // A. ESCUDO CSRF
    $tokenRecibido = $_POST['csrf'] ?? '';
    if (!hash_equals($_SESSION['csrf'], $tokenRecibido)) {
        http_response_code(403);
        echo "<h1 style='color:red;'>Error 403: CSRF inválido</h1>";
        exit;
    }

    // B. RECOGIDA DE DATOS
    $title = trim($_POST['title'] ?? '');
    $artist = trim($_POST['artist'] ?? '');
    $category = $_POST['category'] ?? '';
    $duration = $_POST['duration'] ?? '';
    $src = $_POST['src'] ?? '';

    // C. VALIDACIONES (Aduana)
    if ($title === '') $errores['title'] = 'El título es obligatorio.';
    if ($artist === '') $errores['artist'] = 'El artista es obligatorio.';

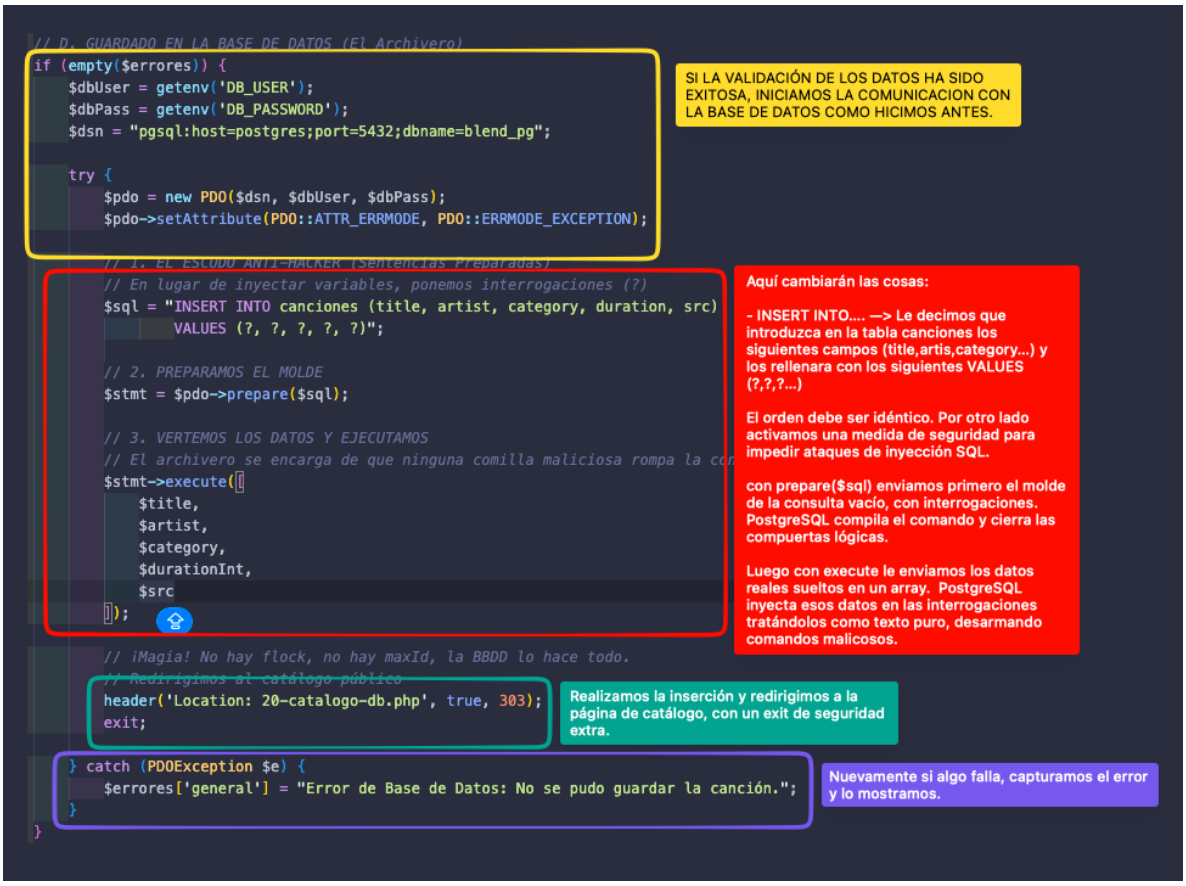
    $allowedCategories = ['electronica', 'rock', 'relax'];
    if (!in_array($category, $allowedCategories, true)) {
        $errores['category'] = 'Categoría no válida.';
    }

    $durationInt = filter_var($duration, FILTER_VALIDATE_INT);
    if ($durationInt === false || $durationInt < 1 || $durationInt > 1800) {
        $errores['duration'] = 'Debe ser entre 1 y 1800 segundos.';
    }

    $allowedSrc = ['audio/demo-01.mp3', 'audio/demo-02.mp3'];
    if (!in_array($src, $allowedSrc, true)) {
        $errores['src'] = 'Audio no válido.';
    }
```

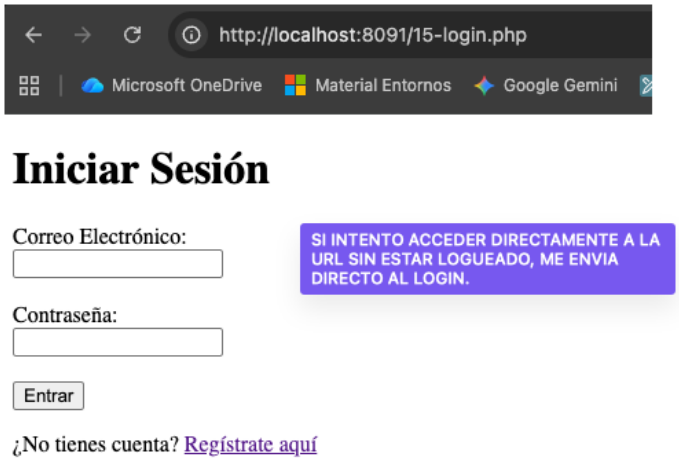
REPETIMOS LAS MEDIDAS DE SEGURIDAD CONOCIDAS

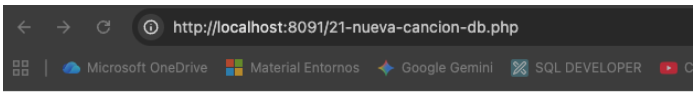
REPETIMOS LA REGOGIDA DE DATOS Y VALIDACIONES YA UTILIZADA



La parte visual es la misma que el formulario anterior, por lo que no se desarrolla su explicación.

PRUEBA:





Añadir Canción Directo a PostgreSQL

← Volver a la Home

Título:

Artista:

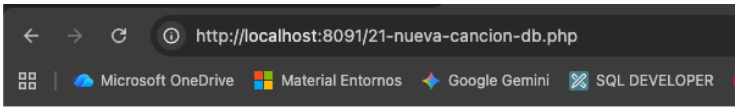
Tenemos acceso a esta seccion solo si estamos logueados.

Categoría:
Selecciona...

Duración (segundos):

Archivo de audio:
Selecciona...

Guardar en PostgreSQL



Añadir Canción Directo a PostgreSQL

← Volver a la Home

LAS VALIDACIONES FUNCIONAN CORRECTAMENTE

Título:
El título es obligatorio.

Artista:
El artista es obligatorio.

Categoría:
Selecciona... Categoría no válida.

Duración (segundos):
Debe ser entre 1 y 1800 segundos.

Archivo de audio:
Selecciona... Audio no válido.

Guardar en PostgreSQL



13) CREACIÓN DE FILTROS INTELIGENTES

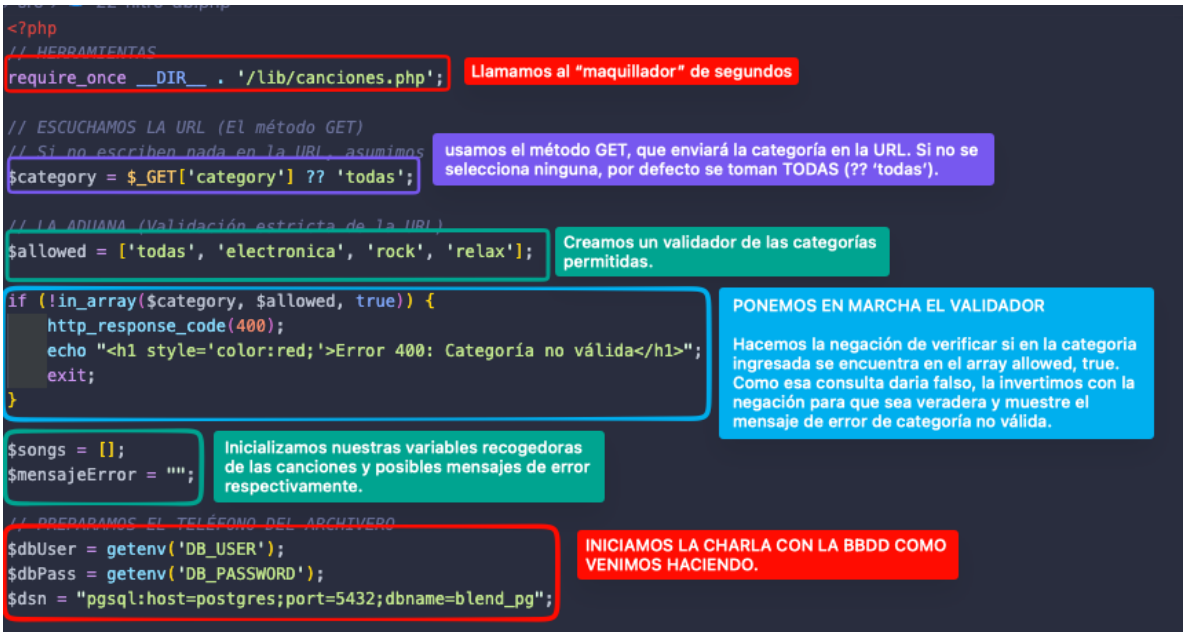
Cuando utilizábamos el archivo canciones.json para almacenar las canciones, creamos un filtro (09-filtro.php).

Con este filtro teníamos que sacar todas las canciones de la caja, llevarlas a PHP y unsar un bucle foreach con un if interno para mirar una por una y preguntar ¿Es ROCK?, si es TRUE la muestro si es FALSE la descarto.

Si ese archivo canciones.json tuviera millones de canciones esta metodología seria inviable.

Como ahora trabajamos con BBDD no sacaremos todos los registros del cajon, le damos una orden precisa a POSTGRESQL → “Busca en el cajón y solo tráeme las canciones que digan “rock” en categoría. Esto se logra con la cláusula WHERE.

Crearemos un nuevo archivo 22-filtro-db.php y veremos que en este caso, utilizaremos el método GET, que enviará la consulta a través de la URL.


Llamamos al "maquillador" de segundos
usamos el método GET, que enviará la categoría en la URL. Si no se selecciona ninguna, por defecto se toman TODAS (?? 'todas').
Creamos un validador de las categorías permitidas.
PONEMOS EN MARCHA EL VALIDADOR
Hacemos la negación de verificar si en la categoria ingresada se encuentra en el array allowed, true. Como esa consulta daria falso, la invertimos con la negación para que sea veradera y muestre el mensaje de error de categoría no válida.
Inicializamos nuestras variables recogedoras de las canciones y posibles mensajes de error respectivamente.
INICIAMOS LA CHARLA CON LA BBDD COMO VENIMOS HACIENDO.


Instanciamos y activamos los errores igual que antes.
DOS POSIBLES CONSULTAS.
1) Si el usuario no ha seleccionado la categoria o si ha elegido "Todas" se prepara la consulta para traer todas las canciones (SELECT * FROM...)
Enviamos la consulta con query y luego usamos el FetchAll para construir el array asociativo.
2) Si el usuario ha ingresado una categoría diferente a todas, construimos esa sentencia con un WHERE.
IMPORTANTE —> La categoría nunca debe escribirse directamente para evitar Inyección SQL. Siempre se escribe un molde (prepare) con un signo de interrogación. Luego en el execute se pasa el dato que reemplazará únicamente a esa interrogación como texto plano, impidiendo la rotura de la estructura SQL.
Finalmente, hacemos un fetchAll para construir el array asociativo.
Si se produce algún error capturamos y mostramos un mensaje para evitar mostrar datos sensibles.

```php
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Filtro BBDD - Blend</title>
</head>
<body>
    <h1>Catálogo Filtrado (PostgreSQL)</h1>

    <h2>Viendo categoría: <span style="color:blue;"><?= htmlspecialchars($category, ENT_QUOTES, 'UTF-8') ?></span></h2>

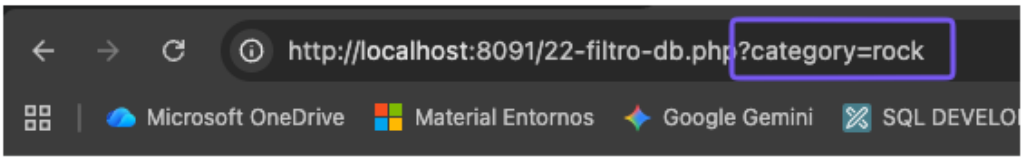
    <!-- Menú de navegación rápida -->
    <nav style="margin-bottom: 20px;">
        <a href="?category=todas">Ver Todas</a> |
        <a href="?category=rock">Rock</a> |
        <a href="?category=electronica">Electrónica</a> |
        <a href="?category=relax">Relax</a>
    </nav>

    <?php if ($mensajeError): ?>
        <p style="color:red;"><strong><?= $mensajeError ?></strong></p>
    <?php endif; ?>

    <!-- PINTAMOS LOS RESULTADOS -->
    <?php if (empty($songs) && !$mensajeError): ?>
        <p>No se encontraron canciones para esta categoría.</p>
    <?php elseif (!empty($songs)): ?>
        <ul>
            <?php foreach ($songs as $song): ?>
                <li>
                    <strong><?= htmlspecialchars($song['title'], ENT_QUOTES, 'UTF-8') ?></strong>
                    (<?= duracionLegible($song['duration']) ?>)
                </li>
            <?php endforeach; ?>
        </ul>
    <?php endif; ?>
</body>
</html>
```

COMO NOVEDAD la parte visual, tendremos esta barra de navegación que añadirá la categoría a la URL enviando asi el filtrado por el metodo GET.

En la URL se añadirá ?category... y en la variable $category se alamcenará lo que va despues del =.



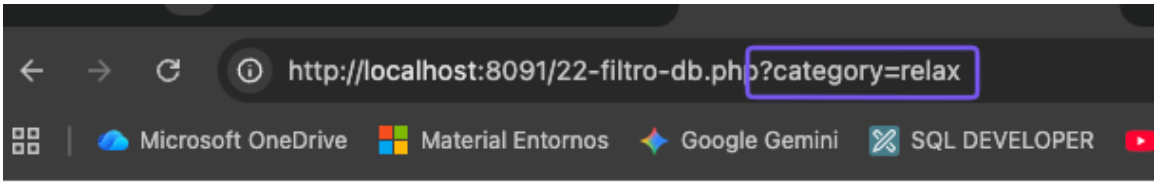
Catálogo Filtrado (PostgreSQL)

Viendo categoría: rock

Ver Todas | Rock | Electrónica | Relax

No se encontraron canciones para esta categoría.

SE ENVIA EN LA URL (METODO GET) PERO NO HAY RESULTADOS PORQUE SOLO HEMOS AÑADIDO DE MOMENTO UNA CANCION A NUESTRA BASE DE DATOS QUE TIENE LA CATEGORIA RELAX.



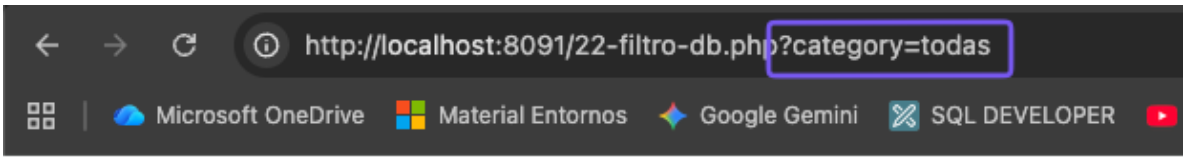
Catálogo Filtrado (PostgreSQL)

Viendo categoría: relax

Ver Todas | Rock | Electrónica | Relax

- **Llamada de emergencia** (4 minutos y 40 segundos)

SE MUESTRA CORRECTAMENTE.



Catálogo Filtrado (PostgreSQL)

Viendo categoría: todas

Ver Todas | Rock | Electrónica | Relax

- **Llamada de emergencia** (4 minutos y 40 segundos)

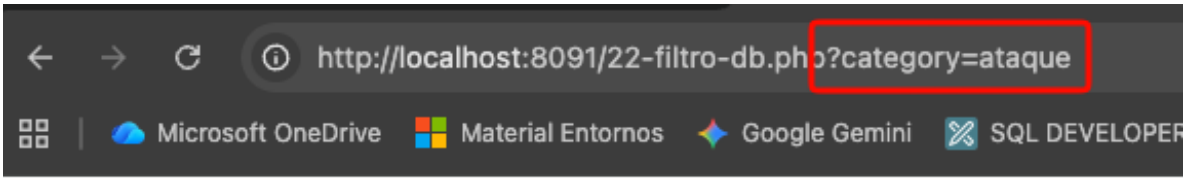
tambien se muestra en todas

LA IMPORTANCIA DEL FILTRADO PREVIO

```php
$allowed = ['todas', 'electronica', 'rock', 'relax'];

if (!in_array($category, $allowed, true)) {
    http_response_code(400);
    echo "<h1 style='color:red;'>Error 400: Categoría no válida</h1>";
    exit;
}
```

Como en el método GET se puede manipular lo que se escribe en la URL, el usuario puede modificar y poner lo que quiera allí. Por eso forzamos una validación previa incluso antes de cualquier consulta en la BBDD → ¿La categoría escrita en la URL está dentro de las permitidas? Si no lo está, directamente mostramos un mensaje de error.



Error 400: Categoría no válida